
Logistic Regression for Multiple Classes

Part 1: The Softmax Regression Model



Previous Binary Classification Setting (2 Classes):

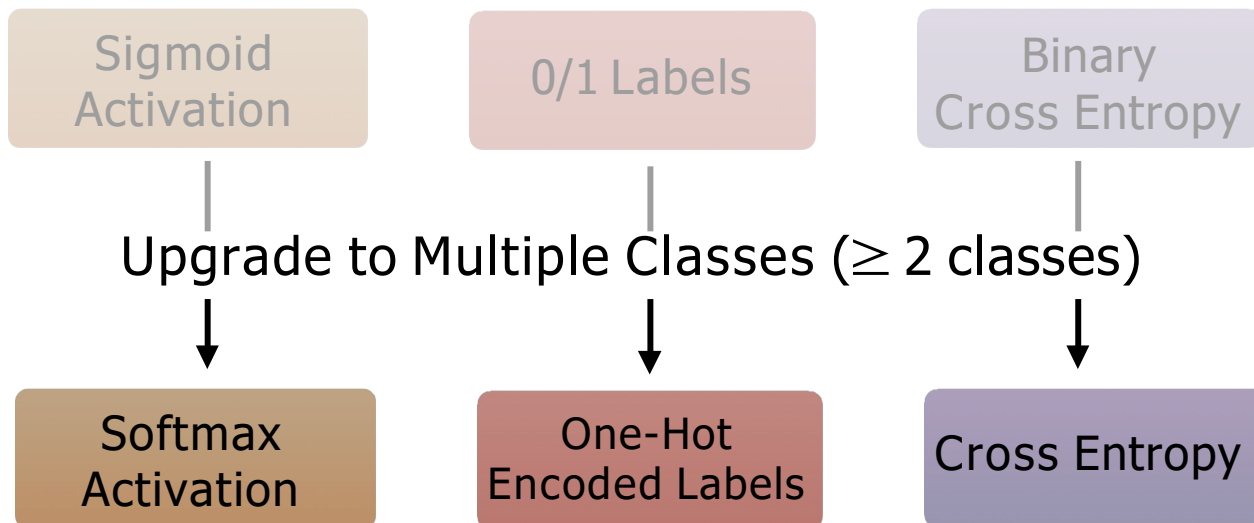
Sigmoid
Activation

0/1 Labels

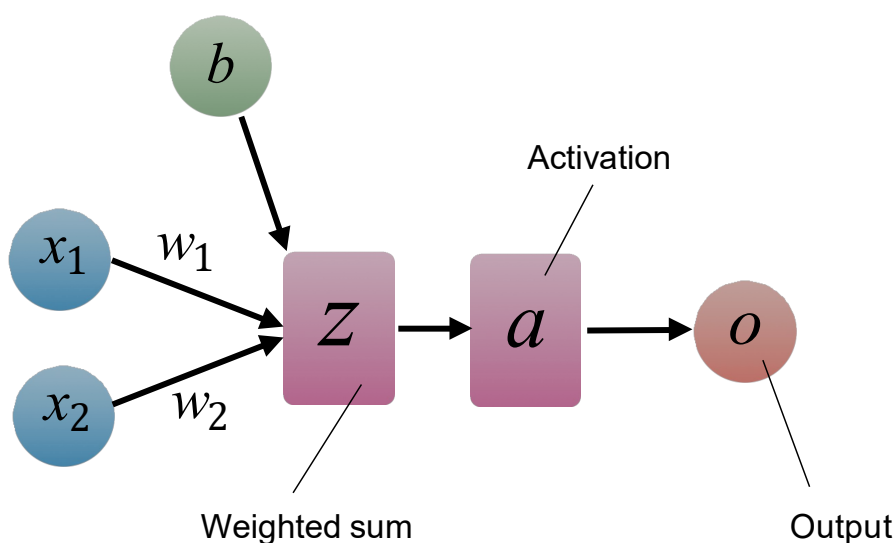
Binary
Cross Entropy



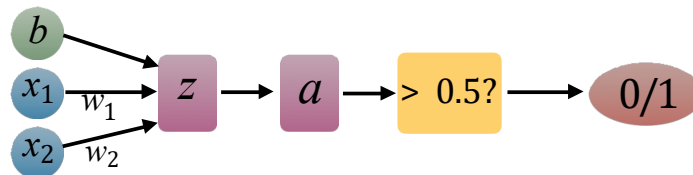
Previous Binary Classification Setting (2 Classes):



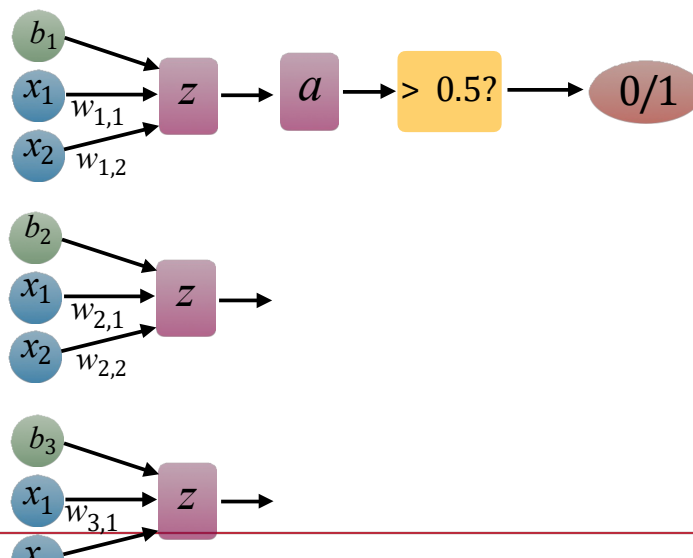
General Structure of Single Layer Neural Nets



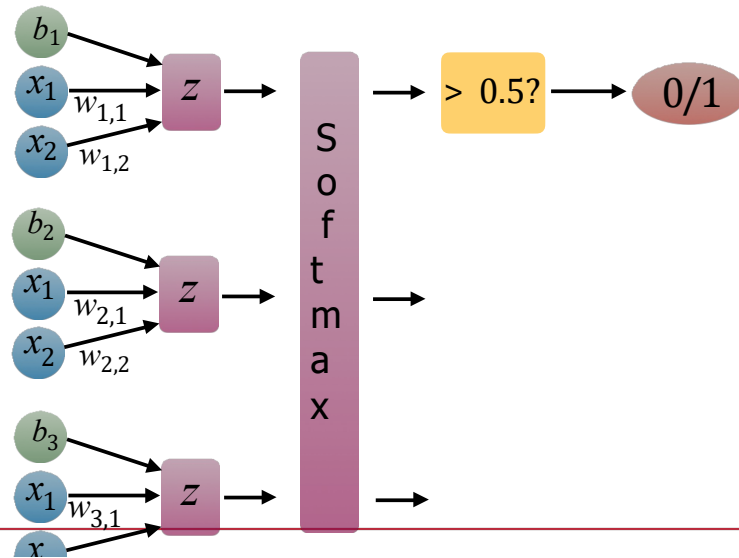
Binary Logistic Regression Model



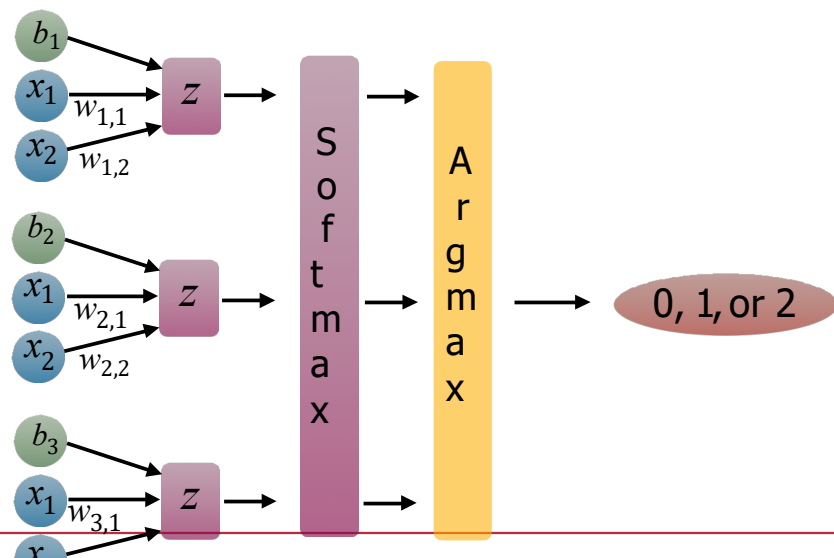
Binary $\rightarrow$ Softmax Regression



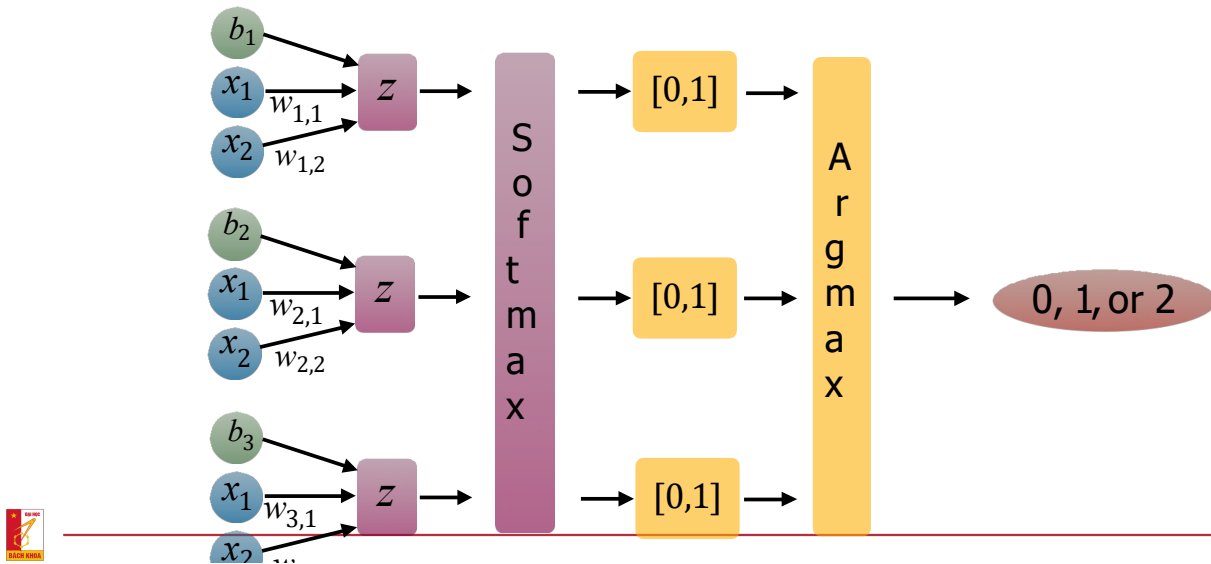
Binary → Softmax Regression



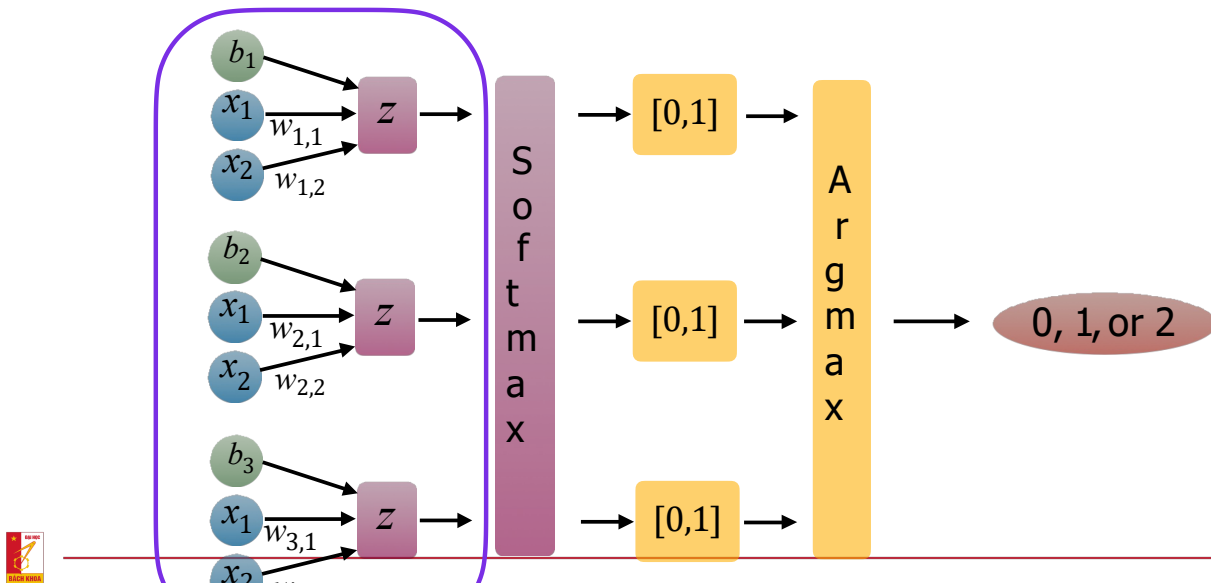
Binary → Softmax Regression



Binary $\rightarrow$ Softmax Regression



Let's focus on this part



Softmax Regression Model

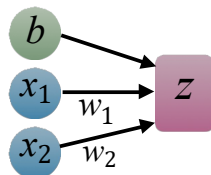
Also "Multinomial" Logistic Regression



Any Code Changes?



Dot Product Plus Bias Unit



x: Single training example

```
import torch
```

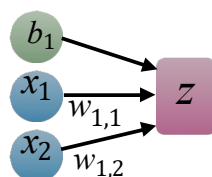
```
x = torch.tensor([1.1, 2.1])  
w = torch.tensor([0.1, 0.2])  
b = torch.tensor(0.5)
```

```
torch.matmul(x, w) + b
```

```
tensor(1.0300)
```



Matrix Multiplication Plus Bias Vector



x: Single training example

```
x = torch.tensor([[1.1, 2.1]])
```

```
w = torch.tensor([[0.1, 0.2],  
                  [0.5, 0.6],  
                  [0.8, 0.9]])
```

```
b = torch.tensor([0.5, 0.6, 0.7])
```

```
torch.matmul(x, w.T) + b
```

```
tensor([[1.0300, 2.4100, 3.4700]])
```



Model Code Changes

Logistic Regression for binary classification (Unit 3)

```
1 import torch
2
3 class LogisticRegression(torch.nn.Module):
4
5     def __init__(self, num_features):
6         super().__init__()
7         self.linear = torch.nn.Linear(num_features, 1)
8
9     def forward(self, x):
10         logits = self.linear(x)
11         probas = torch.sigmoid(logits)
12         return probas
```



Model Code Changes

Softmax regression for multiple class labels

```
1 import torch
2
3 class SoftmaxRegression(torch.nn.Module):
4
5     def __init__(self, num_features):
6         super().__init__()
7         self.linear = torch.nn.Linear(num_features, 3)
8
9     def forward(self, x):
10         logits = self.linear(x)
11         probas = torch.nn.functional.softmax(logits)
12         return probas
```



Model Code Changes

```
logreg.py > LogisticRegression
1 import torch
2
3- class SoftmaxRegression(torch.nn.Module):
4
5     def __init__(self, num_features):
6         super().__init__()
7-         self.linear = torch.nn.Linear(num_features, 3)
8
9     def forward(self, x):
10         logits = self.linear(x)
11-         probas = torch.nn.functional.softmax(logits)
12         return probas

1 import torch
2
3+ class LogisticRegression(torch.nn.Module):
4
5     def __init__(self, num_features):
6         super().__init__()
7+         self.linear = torch.nn.Linear(num_features, 1)
8
9     def forward(self, x):
10         logits = self.linear(x)
11+         probas = torch.sigmoid(logits)
12         return probas
```



Next: From Softmax Scores to Class Labels



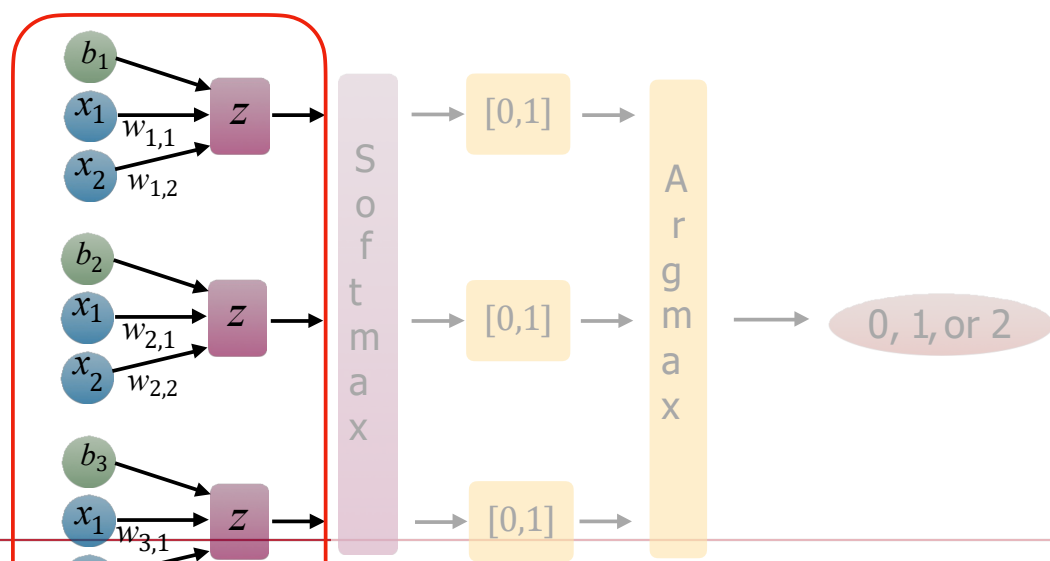
Logistic Regression for Multiple Classes

Part 2: The Softmax Activation Function



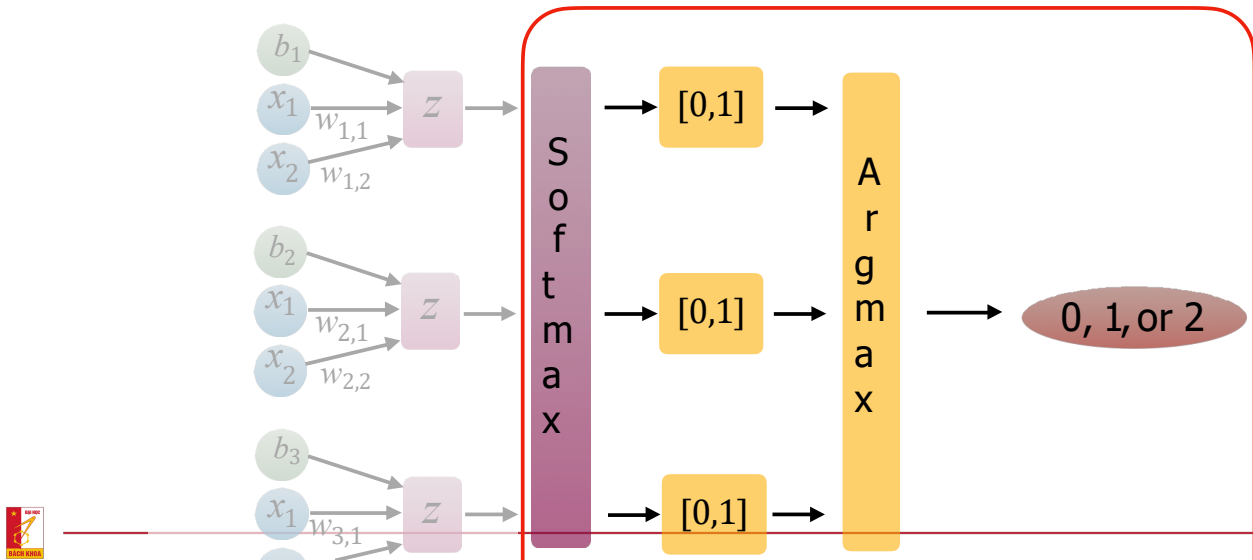
Softmax Regression Model

(Also "Multinomial" Logistic Regression)



Softmax Regression Model

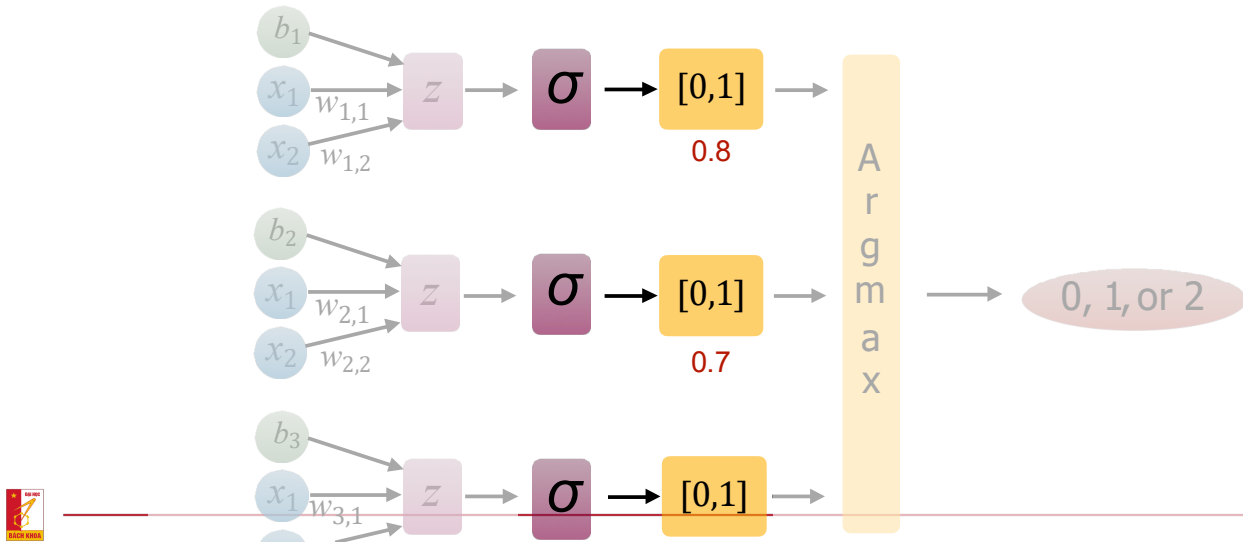
(Also "Multinomial" Logistic Regression)



Softmax is generalized version of the logistic **sigmoid function**

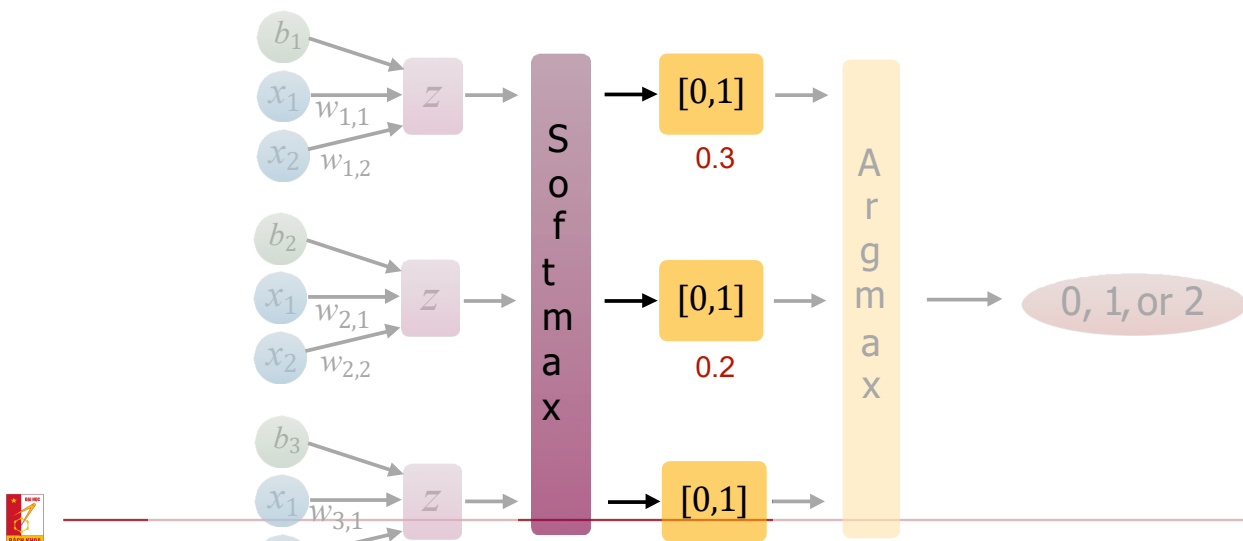
Softmax Regression Model

(Also "Multinomial" Logistic Regression)



Softmax Regression Model

(Also "Multinomial" Logistic Regression)

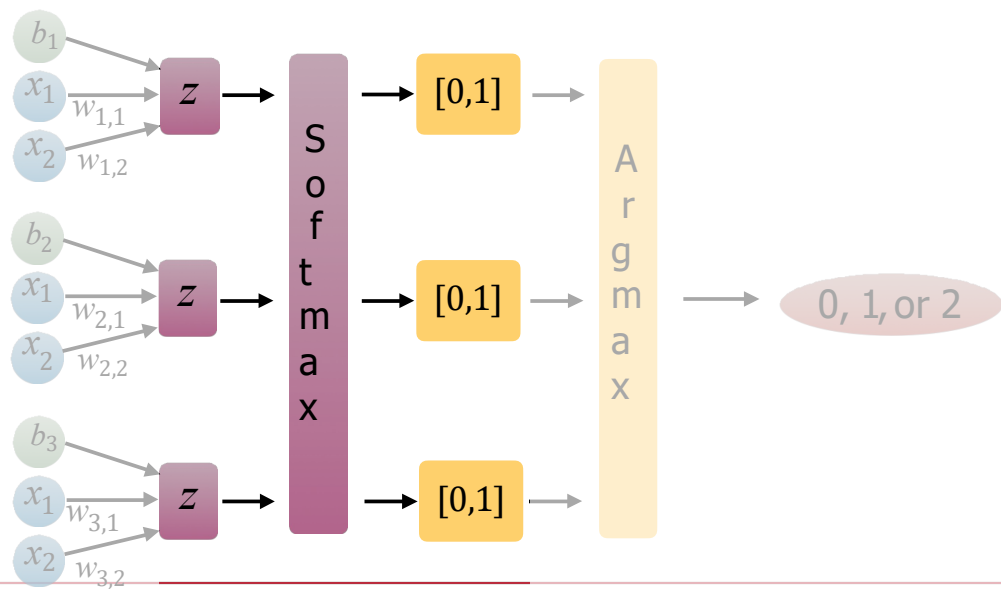


Softmax converts its input values into a probability distribution



A quick look at the Softmax function





$$P(y = t \mid \mathbf{z}_t^{[i]}) = O'_{\text{softmax}}(\mathbf{z}_t^{[i]}) = \frac{e^{z_t^{[i]}}}{\sum_{j=1}^h e^{z_j^{[i]}}}$$

$t \in \{1 \dots h\}$



$$P(y = t \mid z_t^{[i]}) = O'_{\text{softmax}}(z_t^{[i]}) = \frac{e^{z_t^{[i]}}}{\sum_{j=1}^h e^{z_j^{[i]}}}$$

We are looking at a single training example
(i-th training example)



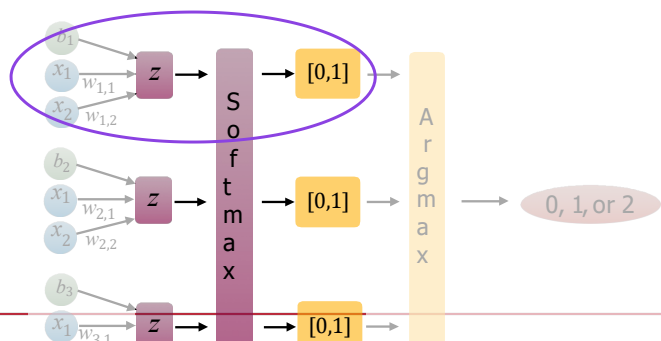
$$P(y = t \mid z_t^{[i]}) = O'_{\text{softmax}}(z_t^{[i]}) = \frac{e^{z_t^{[i]}}}{\sum_{j=1}^h e^{z_j^{[i]}}}$$

Softmax score is a class membership
probability for a label



$$P(y = t \mid z_t^{[i]}) = O'_{\text{softmax}}(z_t^{[i]}) = \frac{e^{z_t^{[i]}}}{\sum_{j=1}^h e^{z_j^{[i]}}}$$

Softmax score for one of the nodes



$$P(y = t \mid z_t^{[i]}) = O'_{\text{softmax}}(z_t^{[i]}) = \frac{e^{z_t^{[i]}}}{\sum_{j=1}^h e^{z_j^{[i]}}}$$

$t \in \{1..h\}$

is the number of class labels

$$P(y = t \mid \mathbf{z}_t^{[i]}) = O'_{\text{softmax}}(\mathbf{z}_t^{[i]}) = \frac{e^{z_t^{[i]}}}{\sum_{j=1}^h e^{z_j^{[i]}}}$$

$t \in \{1 \dots h\}$

Summation over all nodes as a normalization term



2 training examples, 3 classes

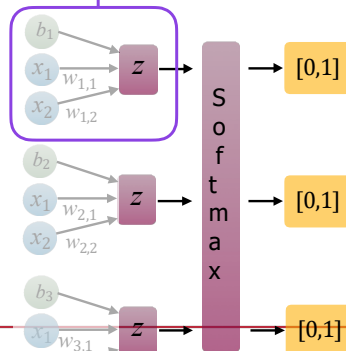
```
import torch
z = torch.tensor([[3.1, -2.3, 5.8], # 1st training example
                  [1.1, 1.9, -8.9]]) # 2nd training example
```



```
import torch

z = torch.tensor([[3.1, -2.3, 5.8], # 1st training example
                  [1.1, 1.9, -8.9]]) # 2nd training example
```

values of class 1



```
import torch

z = torch.tensor([[3.1, -2.3, 5.8], # 1st training example
                  [1.1, 1.9, -8.9]]) # 2nd training example
```

```
import torch.nn.functional as F

F.softmax(z, dim=1)
```

```
tensor([[6.2955e-02, 2.8434e-04, 9.3676e-01],
        [3.1002e-01, 6.8996e-01, 1.4075e-05]])
```

values of class 1 converted to probabilities



```
import torch

z = torch.tensor([[3.1, -2.3, 5.8], # 1st training example
                  [1.1, 1.9, -8.9]]) # 2nd training example

import torch.nn.functional as F

F.softmax(z, dim=1)

tensor([[6.2955e-02, 2.8434e-04, 9.3676e-01],
        [3.1002e-01, 6.8996e-01, 1.4075e-05]])
```

6.3e-02 means
 $6.3 \times 10^{-2} = 0.063$



```
import torch

z = torch.tensor([[3.1, -2.3, 5.8], # 1st training example
                  [1.1, 1.9, -8.9]]) # 2nd training example

import torch.nn.functional as F

F.softmax(z, dim=1)

tensor([[6.2955e-02, 2.8434e-04, 9.3676e-01],
        [3.1002e-01, 6.8996e-01, 1.4075e-05]])
```

For each training example,
columns sum up to one



```
import torch

z = torch.tensor([[3.1, -2.3, 5.8], # 1st training example
                  [1.1, 1.9, -8.9]]) # 2nd training example

import torch.nn.functional as F

F.softmax(z, dim=1)

tensor([[6.2955e-02, 2.8434e-04, 9.3676e-01],
        [3.1002e-01, 6.8996e-01, 1.4075e-05]])

torch.set_printoptions(precision=2, sci_mode=False)

F.softmax(z, dim=1)

tensor([[ 0.06,  0.00,  0.94],
        [ 0.31,  0.69,  0.00]])
```



Next:
From Softmax Activation to Class Labels



Logistic Regression for Multiple Classes

Part 3: From Softmax Scores to Class Labels

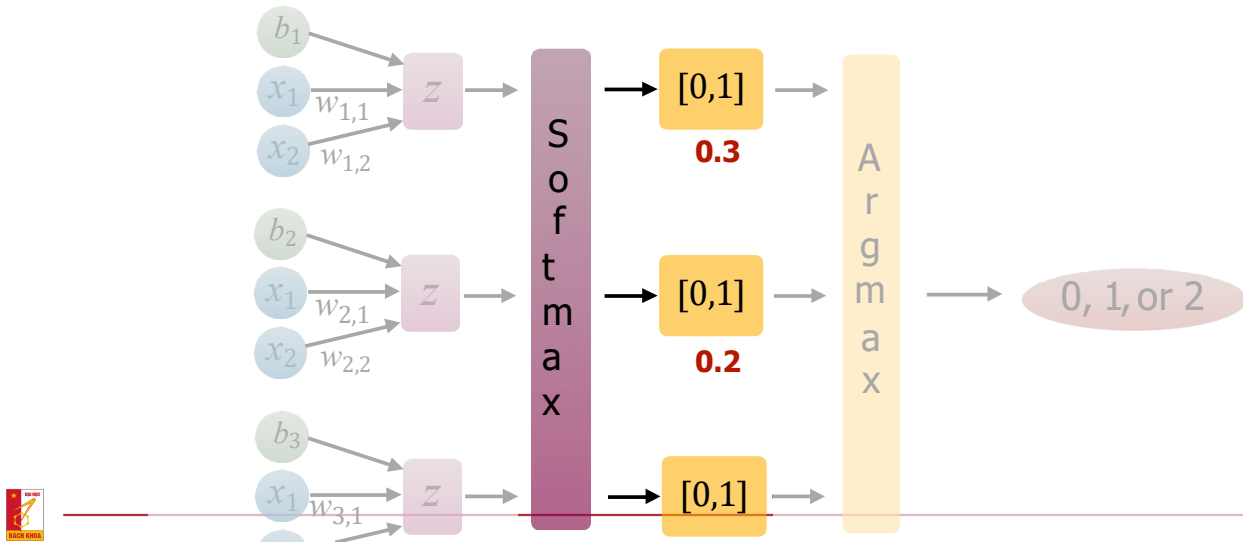


From Softmax Scores to Class Labels

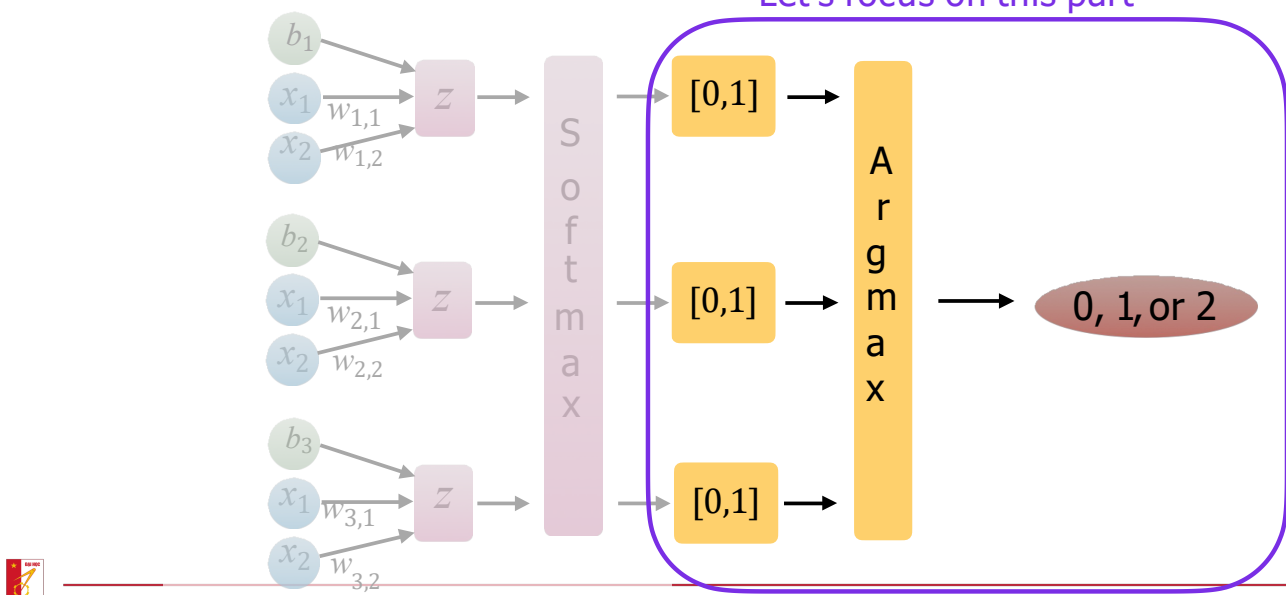


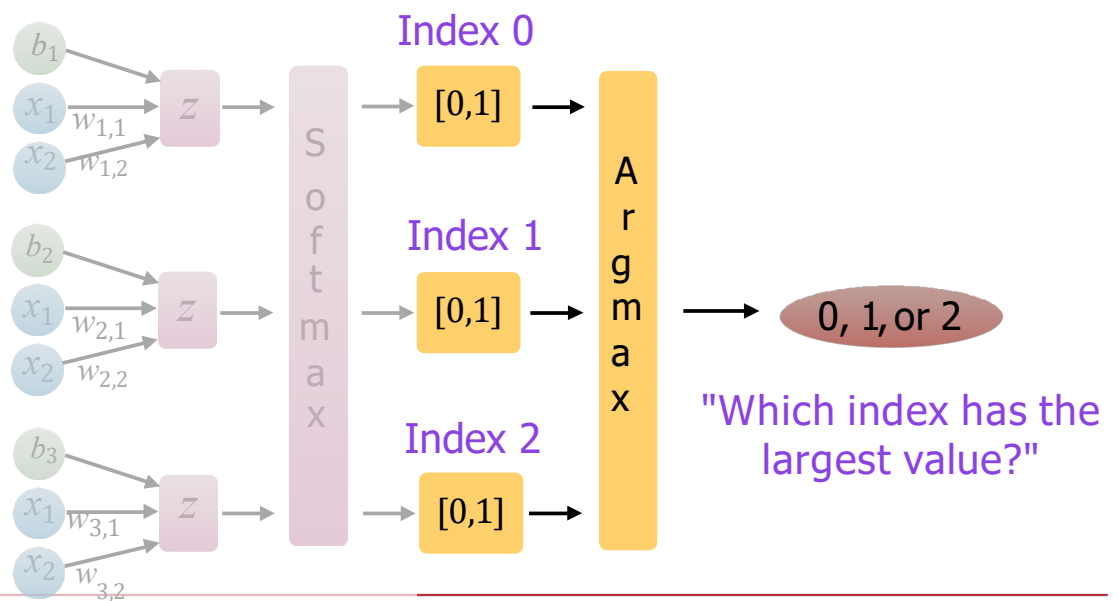
Softmax Regression Model

(Also "Multinomial" Logistic Regression)



Let's focus on this part





```
sm = F.softmax(z, dim=1)
sm
```

```
tensor([[ 0.06,  0.00,  0.94],
        [ 0.31,  0.69,  0.00]])
```

```
sm = F.softmax(z, dim=1)
sm
```

```
tensor([[ 0.06,  0.00,  0.94],
        [ 0.31,  0.69,  0.00]])
```

Example 1

Example 2



What is the **largest score** for each example?



```
sm = F.softmax(z, dim=1)
sm
```

```
tensor([[ 0.06,  0.00,  0.94],
        [ 0.31,  0.69,  0.00]])
```



```
sm = F.softmax(z, dim=1)
sm
```

```
tensor([[ 0.06,  0.00,  0.94],
        [ 0.31,  0.69,  0.00]])
```

Index 2

Index 1



```
sm = F.softmax(z, dim=1)
sm
```

```
tensor([[ 0.06,  0.00,  0.94],
        [ 0.31,  0.69,  0.00]])
```

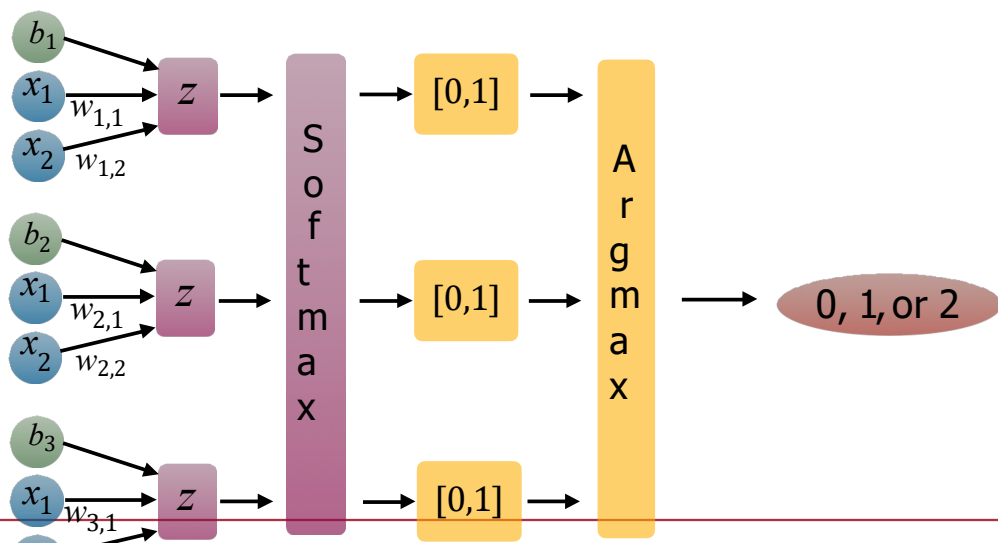
```
torch.argmax(sm, dim=1)
```

```
tensor([2, 1])
```



Softmax Regression Model

(Also "Multinomial" Logistic Regression)



Next: What is the loss function we use for training?



Logistic Regression for Multiple Classes

Part 4: The Cross Entropy Loss Function for Multiple Classes



Previous Unit

Logistic
Regression (LR)

Architecture

0/1 Labels

Classification task

Binary Cross
Entropy

Loss function



Architecture

Logistic
Regression (LR)

Classification task

0/1 Labels

Loss function

Binary Cross
Entropy

Upgrade to Multiple Classes (≥ 2 classes)



Architecture

Classification task

Loss function

Logistic
Regression (LR)

0/1 Labels

Binary Cross
Entropy

Upgrade to Multiple Classes (≥ 2 classes)

Softmax
Regression

One-Hot
Encoded Labels

Cross Entropy



Architecture

Classification task

Loss function

Logistic
Regression (LR)

0/1 Labels

Binary Cross
Entropy

Upgrade to Multiple Classes (≥ 2 classes)

Softmax
Regression

One-Hot
Encoded Labels

Cross Entropy

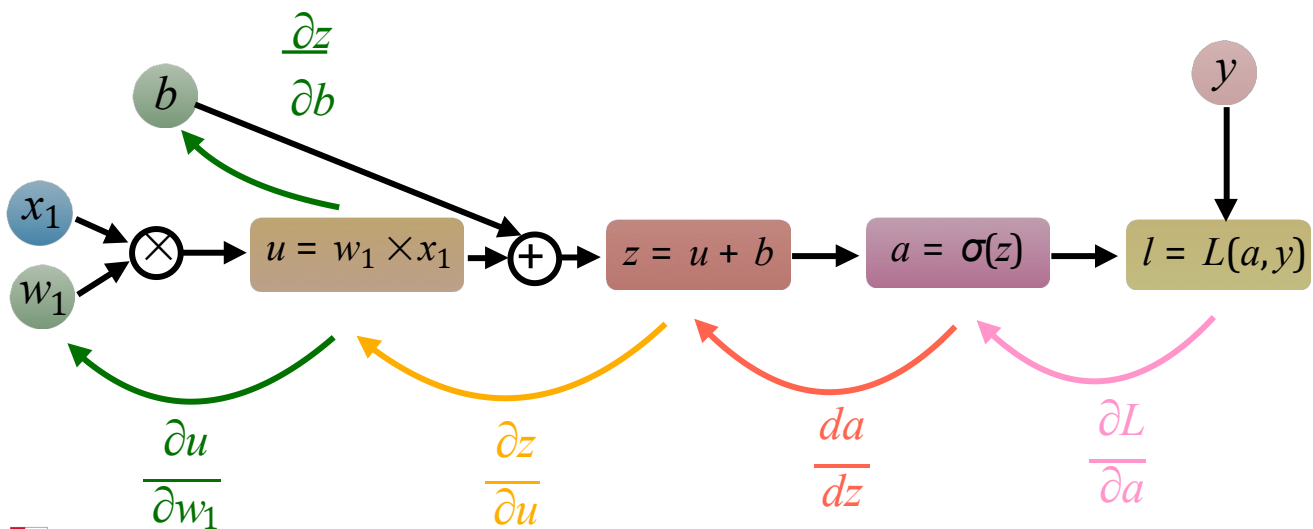
?



Why do we need a loss function?



Parameter Updates (via Stochastic Gradient Descent)



Binary cross entropy loss (Unit 3)

$$L = \frac{1}{n} \sum_{i=1}^n - \left(y^{[i]} \log(a^{[i]}) + (1 - y^{[i]}) \log(1 - a^{[i]}) \right)$$



Binary cross entropy loss for multiple training examples

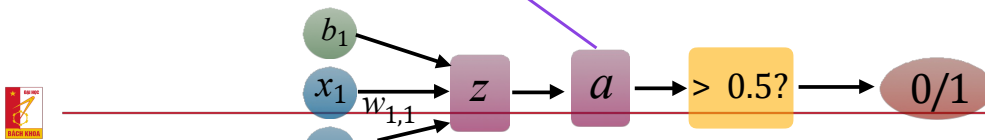
$$L = \frac{1}{n} \sum_{i=1}^n - \left(y^{[i]} \log(a^{[i]}) + (1 - y^{[i]}) \log(1 - a^{[i]}) \right)$$



Binary cross entropy loss for multiple training examples

$$L = \frac{1}{n} \sum_{i=1}^n - \left(y^{[i]} \log(a^{[i]}) + (1 - y^{[i]}) \log(1 - a^{[i]}) \right)$$

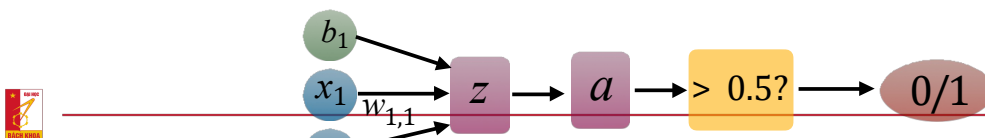
Activation value (logistic sigmoid)



Binary cross entropy loss for multiple training examples

$$L = \frac{1}{n} \sum_{i=1}^n - \left(y^{[i]} \log(a^{[i]}) + (1 - y^{[i]}) \log(1 - a^{[i]}) \right)$$

Class label, 0 or 1



Binary Multi-category cross entropy loss for
multiple training examples and multiple classes

$$L = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K -y_k^{[i]} \log(a_k^{[i]})$$



Binary Multi-category cross entropy loss for
multiple training examples and multiple classes

$$L = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K -y_k^{[i]} \log(a_k^{[i]})$$



Binary Multi-category cross entropy loss for
multiple training examples and multiple classes

$$L = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K -y_k^{[i]} \log(a_k^{[i]})$$



Binary Multi-category cross entropy loss for
multiple training examples and multiple classes

$$L = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K -y_k^{[i]} \log(a_k^{[i]})$$

Assumes a one-hot encoding



Class labels

0
1
3
2

One-hot encoded class labels

Class 0	Class 1	Class 2	Class 3



Class labels

0
1
3
2



One-hot encoded class labels

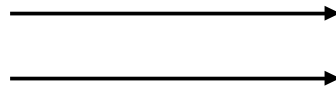
Class 0	Class 1	Class 2	Class 3
1	0	0	0



Class labels

One-hot encoded class labels

0
1
3
2



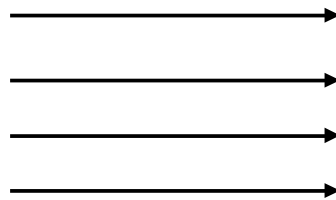
Class 0	Class 1	Class 2	Class 3
1	0	0	0
0	1	0	0



Class labels

One-hot encoded class labels

0
1
3
2



Class 0	Class 1	Class 2	Class 3
1	0	0	0
0	1	0	0
0	0	0	1
0	0	1	0



Binary Multi-category cross entropy loss for
multiple training examples and multiple classes

$$L = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K -y_k^{[i]} \log(a_k^{[i]})$$

Assumes a one-hot encoding
(1's and 0's)



$$L = \frac{1}{n} \sum_{i=1}^n - (y^{[i]} \log(a) + (1 - y^{[i]}) \log(1 - a))$$

0 or 1 0 or 1



$$L = \frac{1}{n} \sum_{i=1}^n - (y^{[i]} \log(a) + (1 - y^{[i]}) \log(1 - a))$$

suppose this is 1



$$L = \frac{1}{n} \sum_{i=1}^n - (y^{[i]} \log(a) + (1 - y^{[i]}) \log(1 - a))$$

suppose this is 1 then this is 0



$$L = \frac{1}{n} \sum_{i=1}^n - \left(\underbrace{y^{[i]}}_{0 \text{ or } 1} \log(a) + (1 - \underbrace{y^{[i]}}_{0 \text{ or } 1}) \log(1 - a) \right)$$

Multi-category cross entropy is a generalization of the binary version
(and both produce the same results for binary case)

$$L = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K - \underbrace{y_k^{[i]}}_{0 \text{ or } 1} \log(a_k^{[i]})$$



Next: Cross Entropy Loss Code Walkthrough

Later: From Softmax Regression to Multilayer Neural Networks



Logistic Regression for Multiple Classes

Part 5: A Cross Entropy Walkthrough With Code



Binary ~~Multi~~-category cross entropy loss for
multiple training examples and multiple classes

$$L = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K -y_k^{[i]} \log(a_k^{[i]})$$



Binary Multi-category cross entropy loss for multiple training examples and multiple classes

$$L = \frac{1}{n} \sum_{i=1}^n \left(\sum_{k=1}^K -y_k^{[i]} \log(a_k^{[i]}) \right)$$

Assumes a one-hot encoding



Class labels

One-hot encoded class labels

		Class 0	Class 1	Class 2
0	→	1	0	0
2	→	0	0	1
2	→	0	0	1
1	→	0	1	0



Let's see how this works out in code



One-hot encoding in PyTorch

```
import torch
import torch.nn.functional as F
```

```
y = torch.tensor([0, 2, 2, 1])
y_onehot = F.one_hot(y)
y_onehot
```

```
tensor([[1, 0, 0],
        [0, 0, 1],
        [0, 0, 1],
        [0, 1, 0]])
```



Net inputs (logits) and Softmax activations

```
net_inputs = torch.tensor([[ 1.5,  0.1, -0.4],
                           [ 0.5,  0.7,  2.1],
                           [-2.1,  1.1,  0.8],
                           [ 1.1,  2.5, -1.2]])
```

Suppose these were computing by our model



Net inputs (logits) and Softmax activations

```
net_inputs = torch.tensor([[ 1.5,  0.1, -0.4],
                           [ 0.5,  0.7,  2.1],
                           [-2.1,  1.1,  0.8],
                           [ 1.1,  2.5, -1.2]])
```

```
activations = torch.softmax(net_inputs, dim=1)
activations
```

```
tensor([[0.7162, 0.1766, 0.1071],
        [0.1394, 0.1702, 0.6904],
        [0.0229, 0.5613, 0.4158],
        [0.1940, 0.7866, 0.0194]])
```



Net inputs (logits) and Softmax activations

↔

```
net_inputs = torch.tensor([[ 1.5,  0.1, -0.4],
                           [ 0.5,  0.7,  2.1],
                           [-2.1,  1.1,  0.8],
                           [ 1.1,  2.5, -1.2]])
```

```
activations = torch.softmax(net_inputs, dim=1)
activations
```

```
tensor([[0.7162, 0.1766, 0.1071],
        [0.1394, 0.1702, 0.6904],
        [0.0229, 0.5613, 0.4158],
        [0.1940, 0.7866, 0.0194]])
```

Means that axis 1 (column-axis)
goes together



Net inputs (logits) and Softmax activations

↔

```
net_inputs = torch.tensor([[ 1.5,  0.1, -0.4],
                           [ 0.5,  0.7,  2.1],
                           [-2.1,  1.1,  0.8],
                           [ 1.1,  2.5, -1.2]])
```

```
activations = torch.softmax(net_inputs, dim=1)
activations
```

```
tensor([[0.7162, 0.1766, 0.1071],
        [0.1394, 0.1702, 0.6904],
        [0.0229, 0.5613, 0.4158],
        [0.1940, 0.7866, 0.0194]])
```

Row (column-elements) sum to 1

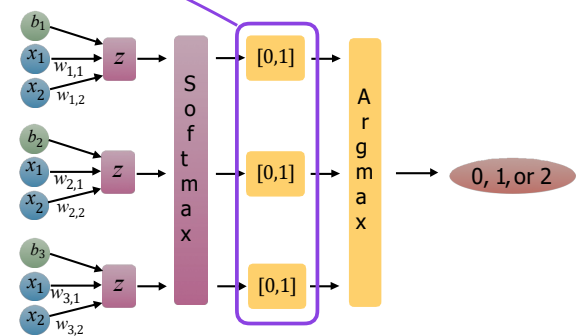


Net inputs (logits) and Softmax activations

```
net_inputs = torch.tensor([[ 1.5,  0.1, -0.4],
                           [ 0.5,  0.7,  2.1],
                           [-2.1,  1.1,  0.8],
                           [ 1.1,  2.5, -1.2]])

activations = torch.softmax(net_inputs, dim=1)
activations

tensor([[0.7162, 0.1766, 0.1071],
        [0.1394, 0.1702, 0.6904],
        [0.0229, 0.5613, 0.4158],
        [0.1940, 0.7866, 0.0194]])
```



```
activations = torch.softmax(net_inputs, dim=1)
activations

tensor([[0.7162, 0.1766, 0.1071],
        [0.1394, 0.1702, 0.6904],
        [0.0229, 0.5613, 0.4158],
        [0.1940, 0.7866, 0.0194]])
```

```
y = torch.tensor([0, 2, 2, 1])
y_onehot = F.one_hot(y)
y_onehot
```

```
tensor([[1, 0, 0],
        [0, 0, 1],
        [0, 0, 1],
        [0, 1, 0]])
```

$$L = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K -y_k^{[i]} \log(a_k^{[i]})$$

Training example 1
($i = 1$)

$$\begin{aligned} & -1 \cdot \log(0.7162) \\ & -0 \cdot \log(0.1766) \\ & -0 \cdot \log(0.1071) \\ & = -1 \cdot \log(0.7162) \\ & = 0.333796... \end{aligned}$$



```
activations = torch.softmax(net_inputs, dim=1)
activations
```

```
tensor([[0.7162, 0.1766, 0.1071],
        [0.1394, 0.1702, 0.6904],
        [0.0229, 0.5613, 0.4158],
        [0.1940, 0.7866, 0.0194]])
```

```
y = torch.tensor([0, 2, 2, 1])
y_onehot = F.one_hot(y)
y_onehot
```

```
tensor([[1, 0, 0],
        [0, 0, 1],
        [0, 0, 1],
        [0, 1, 0]])
```



$$L = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K -y_k^{[i]} \log(a_k^{[i]})$$

Training example 2
($i = 2$)

$$\begin{aligned} & -0 \cdot \log(0.1394) \\ & -0 \cdot \log(0.1702) \\ & -1 \cdot \log(0.6904) \\ & = -1 \cdot \log(0.6904) \\ & = 0.370484.. \end{aligned}$$

```
activations = torch.softmax(net_inputs, dim=1)
activations
```

```
tensor([[0.7162, 0.1766, 0.1071],
        [0.1394, 0.1702, 0.6904],
        [0.0229, 0.5613, 0.4158],
        [0.1940, 0.7866, 0.0194]])
```

```
y = torch.tensor([0, 2, 2, 1])
y_onehot = F.one_hot(y)
y_onehot
```

```
tensor([[1, 0, 0],
        [0, 0, 1],
        [0, 0, 1],
        [0, 1, 0]])
```



$$L = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K -y_k^{[i]} \log(a_k^{[i]})$$

Training example 3
($i = 3$)

$$\begin{aligned} & -0 \cdot \log(0.0229) \\ & -0 \cdot \log(0.5613) \\ & -1 \cdot \log(0.4158) \\ & = -1 \cdot \log(0.4158) \\ & = 0.877551... \end{aligned}$$

```
activations = torch.softmax(net_inputs, dim=1)
activations
```

```
tensor([[0.7162, 0.1766, 0.1071],
        [0.1394, 0.1702, 0.6904],
        [0.0229, 0.5613, 0.4158],
        [0.1940, 0.7866, 0.0194]])
```

```
y = torch.tensor([0, 2, 2, 1])
y_onehot = F.one_hot(y)
y_onehot
```

```
tensor([[1, 0, 0],
        [0, 0, 1],
        [0, 0, 1],
        [0, 1, 0]])
```

$$L = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K -y_k^{[i]} \log(a_k^{[i]})$$

Training example 4
($i = 4$)

$$\begin{aligned} & -0 \cdot \log(0.1940) \\ & -1 \cdot \log(0.7866) \\ & -0 \cdot \log(0.1940) \\ & = -1 \cdot \log(0.7866) \\ & = 0.240035... \end{aligned}$$



$$L = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K -y_k^{[i]} \log(a_k^{[i]})$$

Summing over the loss for each training example

$$\begin{aligned} & 1/4 (0.3338 + 0.3705 + 0.8776 + 0.2400) \\ & = 0.4555 \end{aligned}$$



Computing the Cross Entropy Loss in PyTorch



```
import torch
import torch.nn.functional as F

def manual_cross_entropy(net_inputs, y):
    activations = torch.softmax(net_inputs, dim=1)
    y_onehot = F.one_hot(y)
    train_losses = - torch.sum(torch.log(activations) * (y_onehot), dim=1)
    avg_loss = torch.mean(train_losses)
    return avg_loss
```

```
manual_cross_entropy(net_inputs, y)
```

```
tensor(0.4555)
```



We do not need the **one-hot encoding** step
Again, PyTorch keeps care of this for us internally)

We also don't need the **softmax step**
(Again, PyTorch keeps care of this for us internally)



```
import torch
import torch.nn.functional as F

y = torch.tensor([0, 2, 2, 1])
# y_onehot = F.one_hot(y)

net_inputs = torch.tensor([[ 1.5,  0.1, -0.4],
                           [ 0.5,  0.7,  2.1],
                           [-2.1,  1.1,  0.8],
                           [ 1.1,  2.5, -1.2]])

# activations = torch.softmax(net_inputs, dim=1)

F.cross_entropy(net_inputs, y)

tensor(0.4555)
```



```
import torch
import torch.nn.functional as F

y = torch.tensor([0, 2, 2, 1])
# y_onehot = F.one_hot(y)

net_inputs = torch.tensor([[ 1.5,  0.1, -0.4],
                           [ 0.5,  0.7,  2.1],
                           [-2.1,  1.1,  0.8],
                           [ 1.1,  2.5, -1.2]])

# activations = torch.softmax(net_inputs, dim=1)
F.cross_entropy(net_inputs, y)
```

tensor(0.4555)

This is really all we need



Next: From Softmax Regression to Multilayer Neural Networks



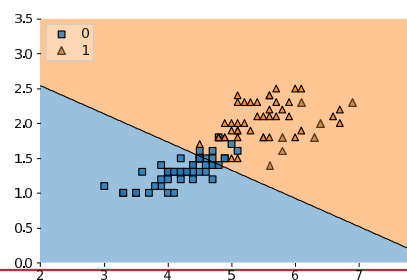
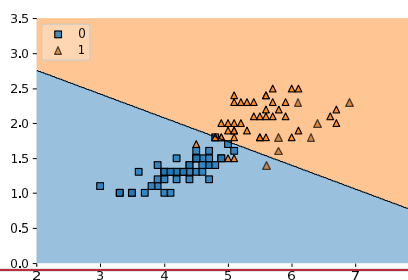
Multilayer Neural Networks And Why We Need Them

Part 1: Looking Beyond Linear Decision Boundaries



Perceptron

Limitation:
never converges on non-linearly separable data



Perceptron

Differentiable loss
+ (stochastic) gradient descent

Logistic
Regression

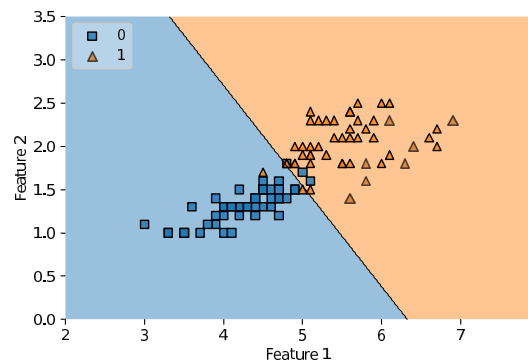
Converges to
loss minimum



Logistic
Regression

Converges to loss
minimum

Limitation:
Can only handle 2 classes!



Logistic
Regression

Multiple output nodes
+ cross entropy loss



Softmax
Regression

Converges to loss
minimum

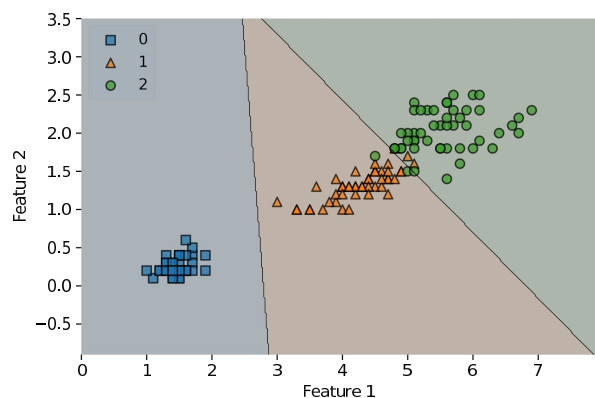
Limitation:
Can only handle 2 classes!



Softmax
Regression

Handles multiple classes!

Limitation: linear
boundaries



Softmax
Regression

Add hidden layers

Multilayer
perceptron

Handles multiple classes!

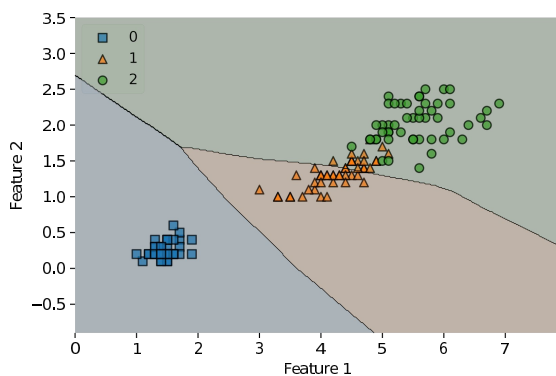
Limitation: linear
boundaries



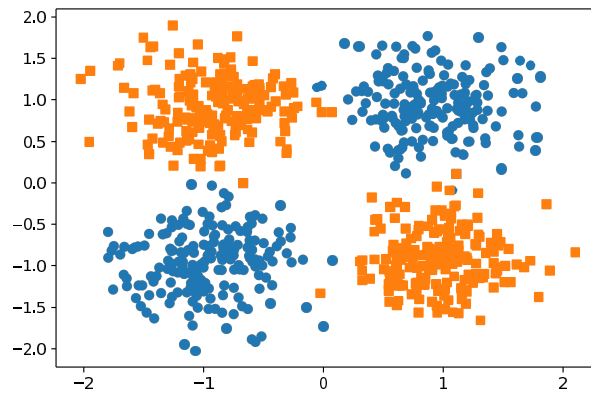
Multilayer
perceptron

Can generate complex
boundaries!

Limitation: Needs more data
and tuning to perform well

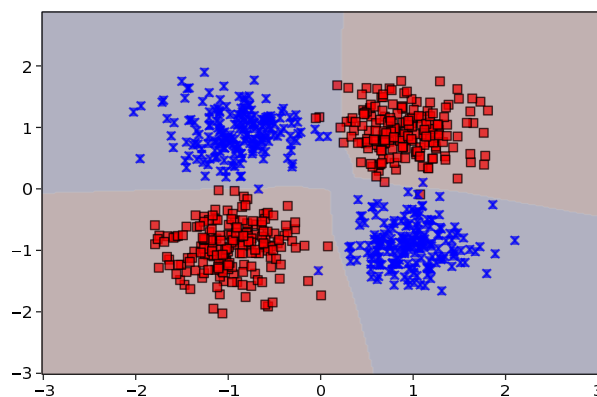


XOR problem



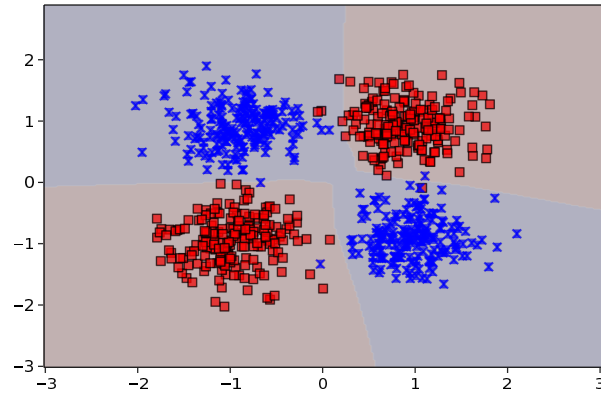
A simple problem logistic
regression cannot solve

XOR problem



A simple problem logistic
regression cannot solve

XOR problem



A problem MLPs can solve!



Logistic / Softmax Regression
is a great baseline!



Next: A closer look at the **multilayer perceptron** model

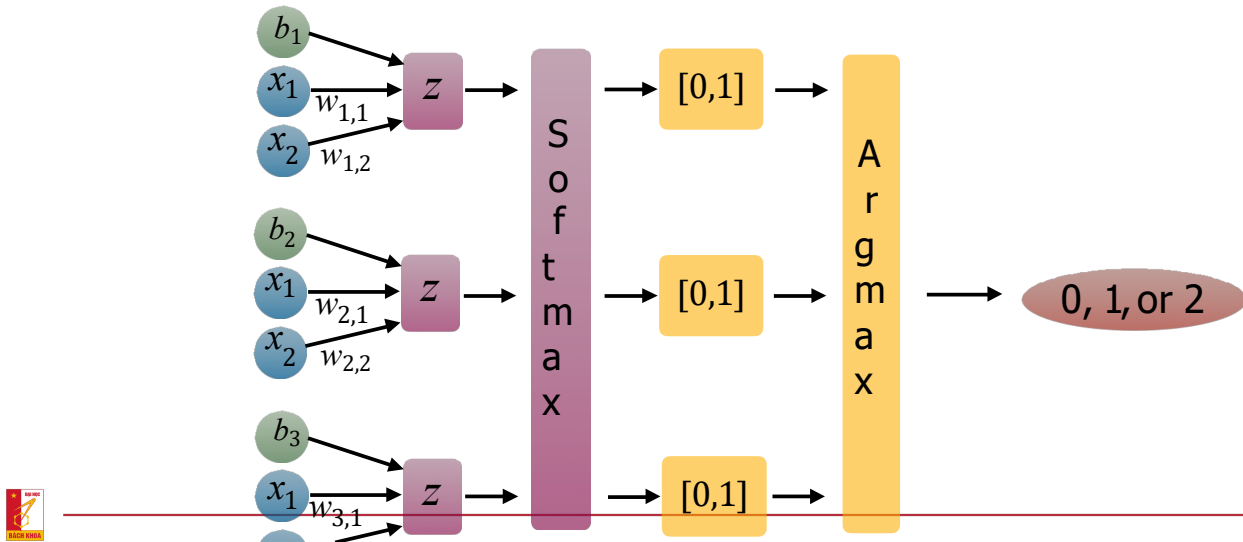


Multilayer Neural Networks And Why We Need Them

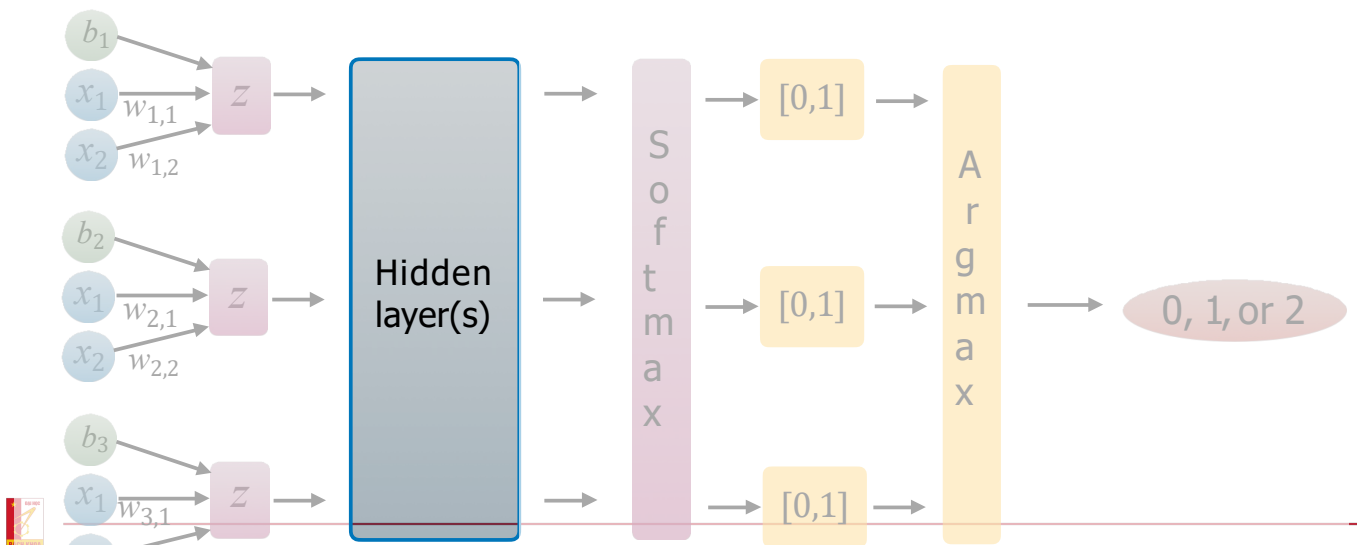
Part 2: The Multilayer Perceptron Architecture



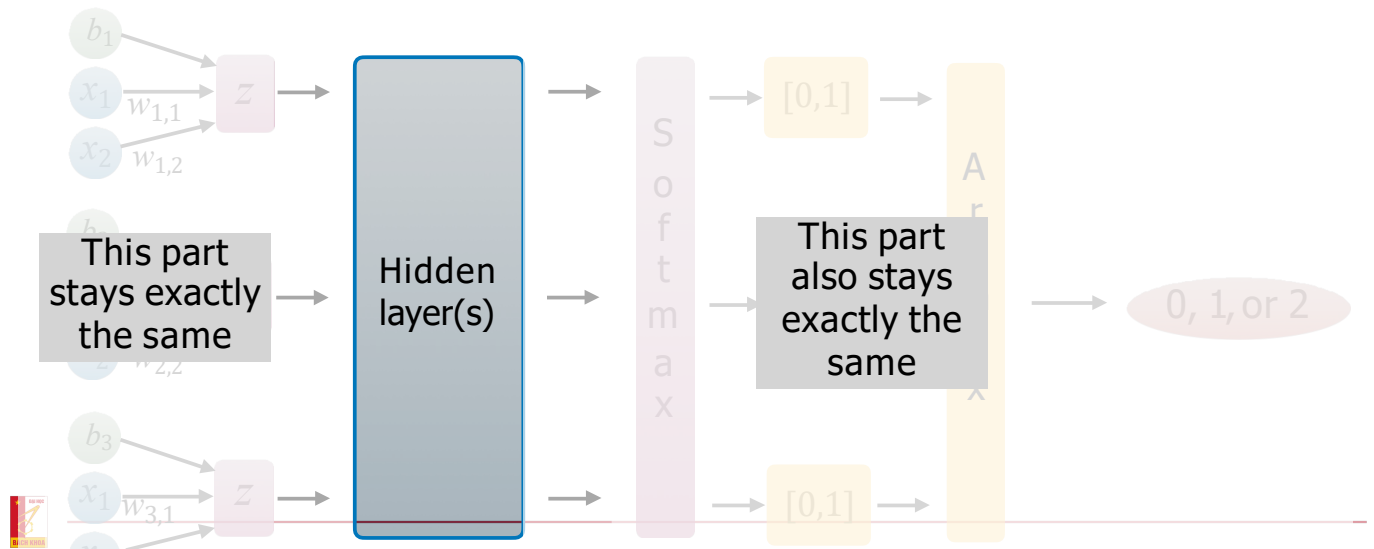
Softmax Regression Model



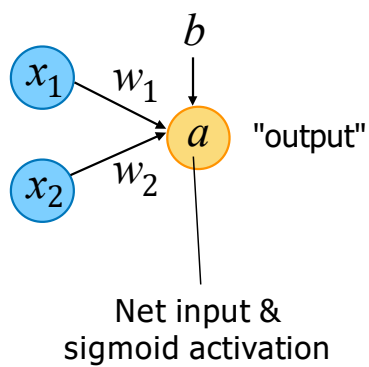
Multilayer Perceptron Model



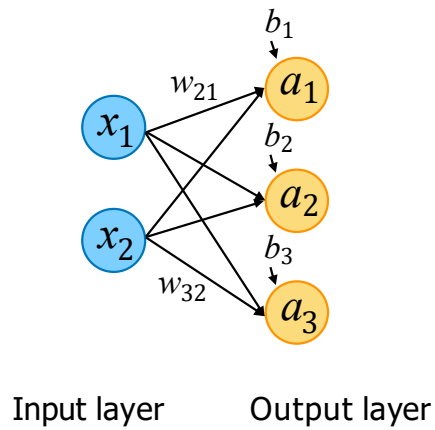
Multilayer Perceptron Model



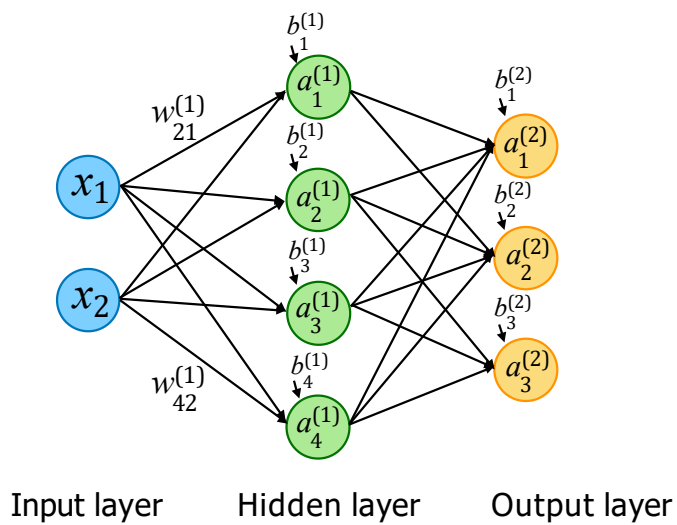
Logistic Regression (Ball-And-Stick Model)



Softmax Regression



Multilayer Perceptron

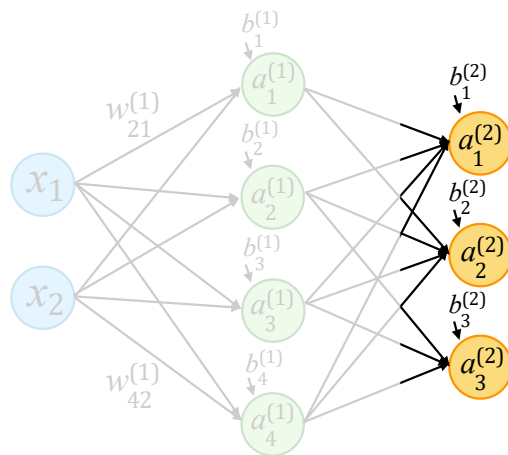


**A multilayer perceptron is
a **fully-connected**
feedforward neural networks**



Activation Function





Softmax activation and multiple nodes for multi-class settings

(like softmax regression)

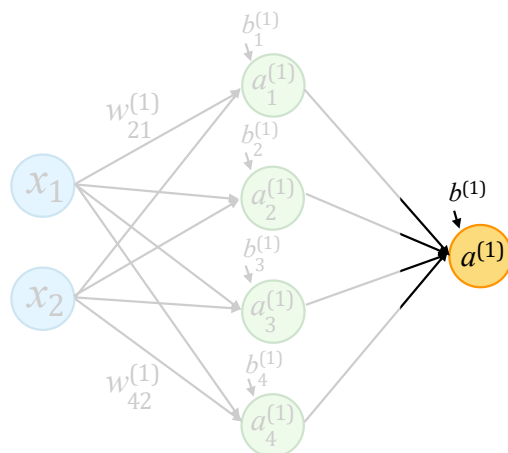
Input layer

Hidden layer

Output layer



Multilayer Perceptron



Sigmoid activation and 1 node for binary classification settings

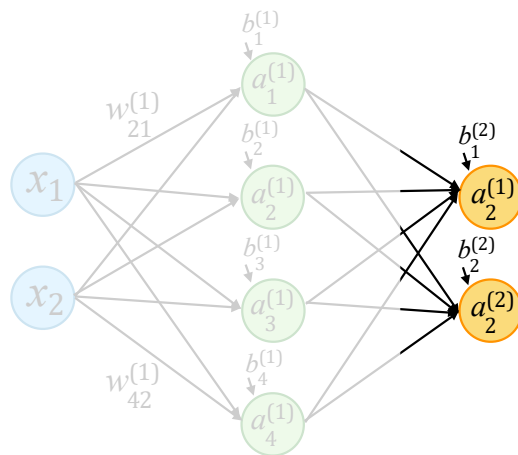
(like logistic regression)

Input layer

Hidden layer

Output layer





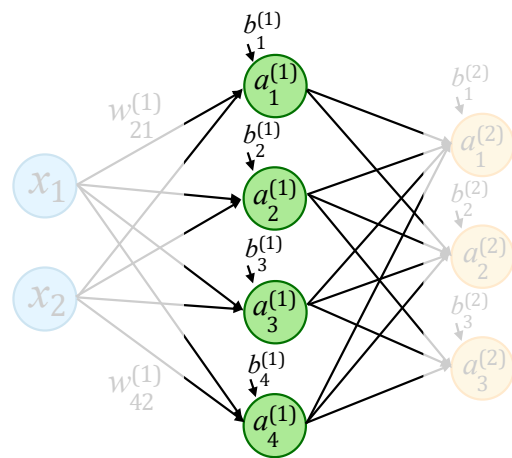
Common in practice:
 Softmax activation and 2 nodes
 for binary classification settings

Input layer Hidden layer Output layer



Hidden layer activation functions



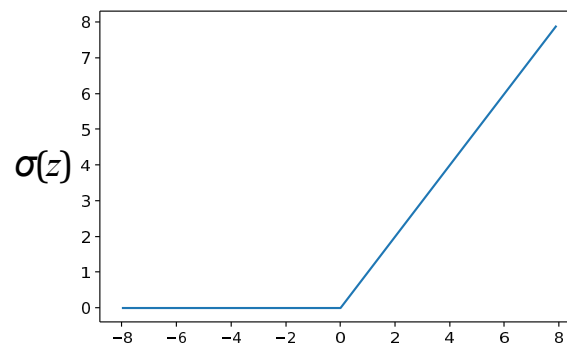


Input layer Hidden layer Output layer

Classic: Logistic sigmoid
Most common: ReLU

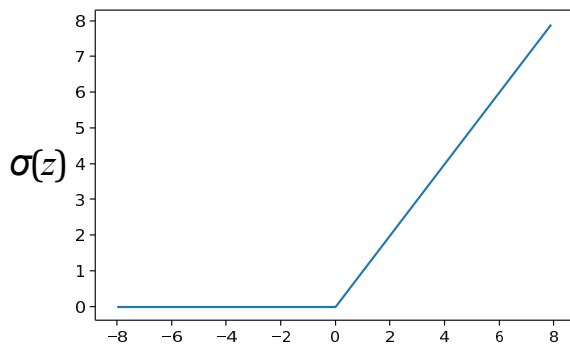


ReLU (Rectified Linear Unit)



ReLU

(Rectified Linear Unit)



$$\text{ReLU}(z) = \begin{cases} z, & \text{if } z \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

$$\text{ReLU}(z) = \max(0, z)$$



Next: Basic architecture design considerations of
multilayer perceptrons



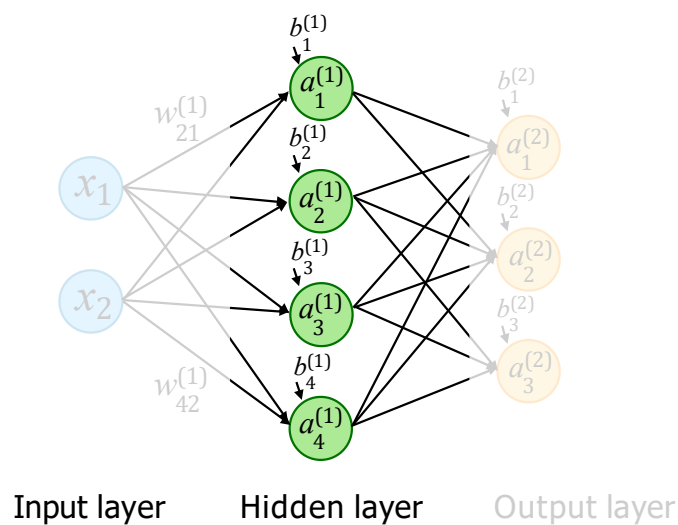
4.2

Multilayer Neural Networks And Why We Need Them Part 3: Basic Architecture Design Considerations



Sebastian Raschka and the Lightning AI Team

Nonlinear Activation Functions



Classic: Logistic sigmoid

Design Consideration #1:

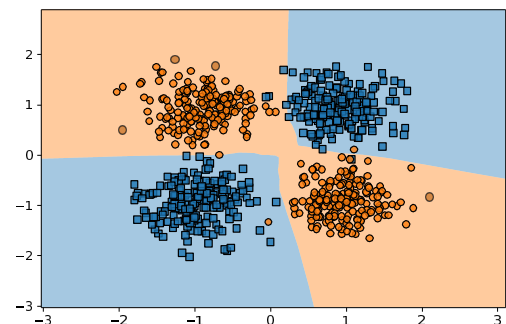
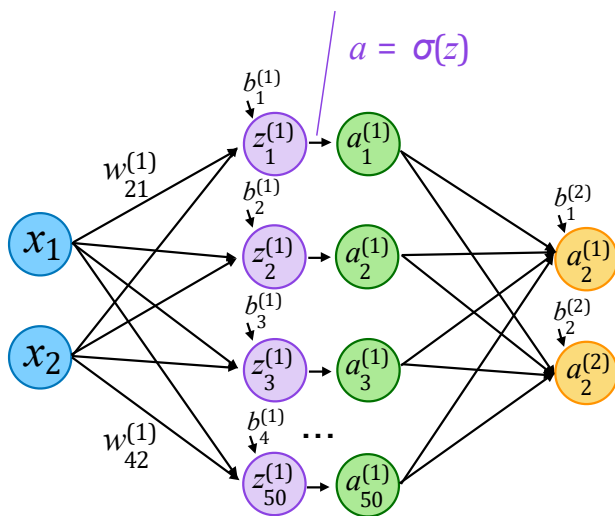
Do we need nonlinear activation functions?



ReLU activation

$$a = \sigma(z)$$

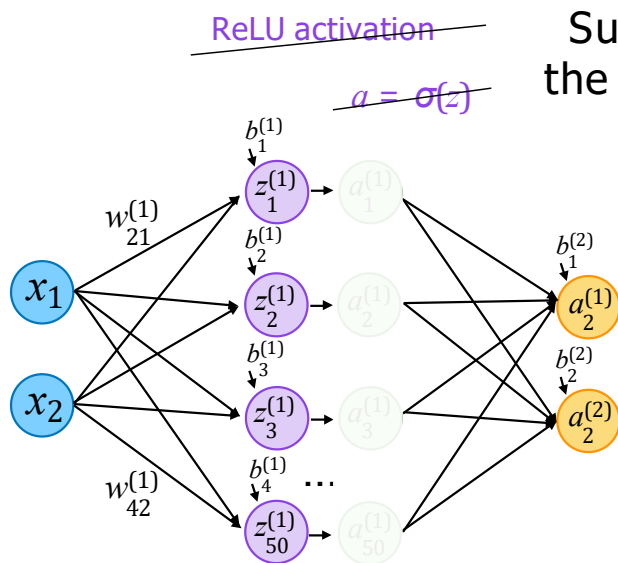
A simple multilayer perceptron



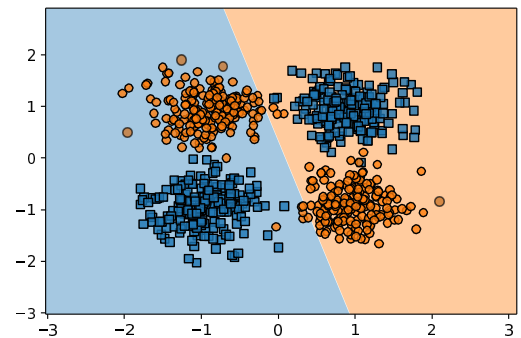
Input layer

Hidden layer

Output layer



Suppose we get rid of the activation function ...



Input layer Hidden layer Output layer

Why do we need hidden layer activations?

Because combination of linear functions is (still) a linear function

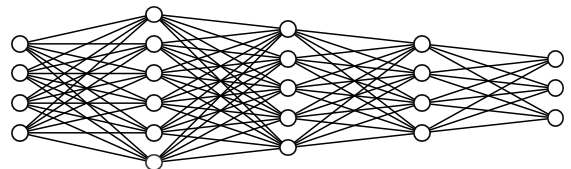
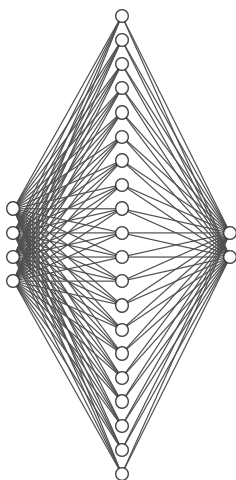


Design Consideration #2:

Wide Vs Deep Networks



Wide Vs Deep Networks

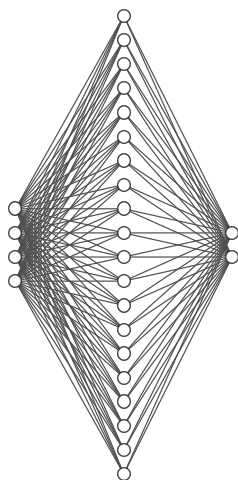


Wide & shallow

Narrow & deep

Wide Vs Deep Networks

In theory, a multilayer perceptron can approximate any function



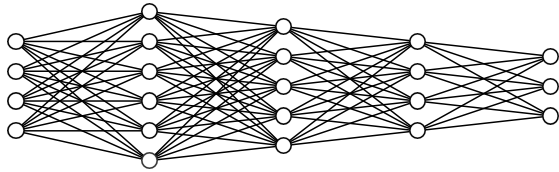
A sufficiently wide MLP with 1 hidden layer can approximate an arbitrary function (given enough training data)

Downsides:

- Needs lots of hidden units
- Is prone to memorization (vs learning)



Wide & shallow



Narrow & deep

Advantages:

- Needs fewer units (nodes)
- Generalizes better (abstract features; see CNN)

Downsides:

- Vanishing/exploding gradient problems
- Harder to train

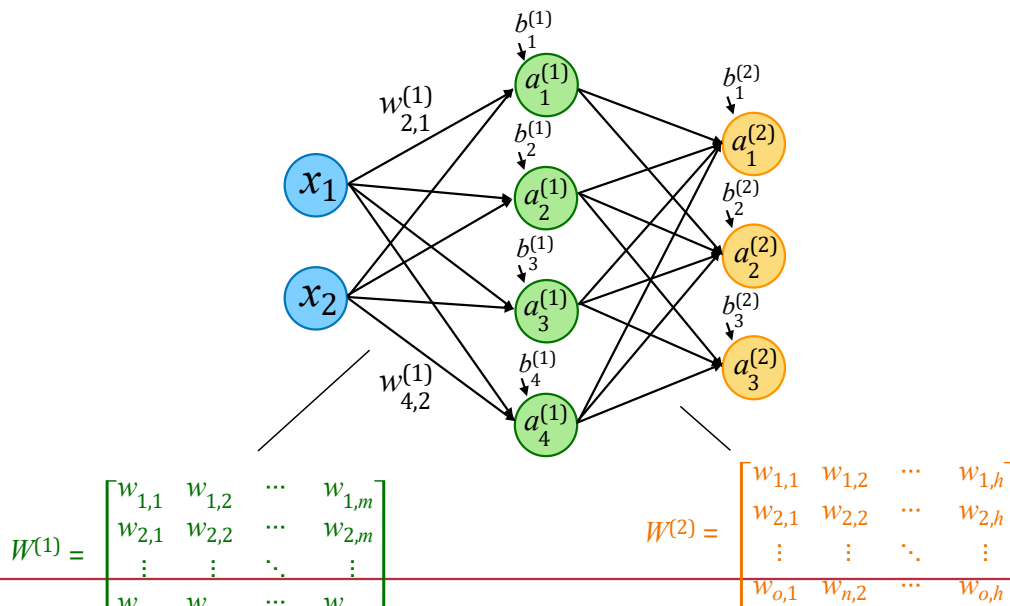


Design Consideration #3:

How do we initialize the weights?

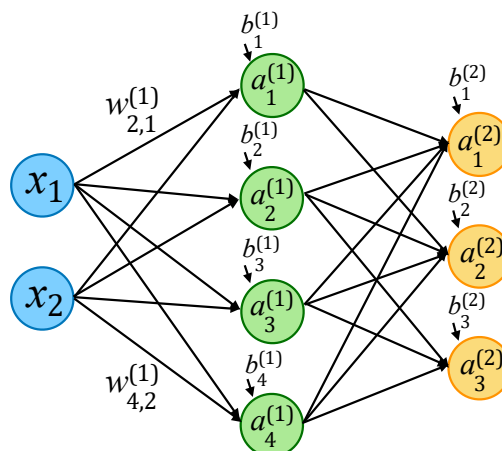


Random Weight Initialization



Random Weight Initialization

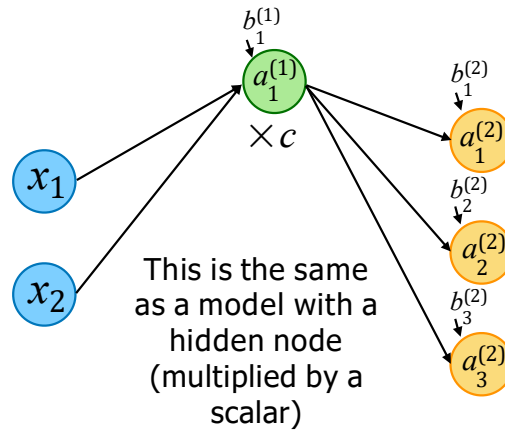
Thought experiment:
Imagine all initial weights are 0
(similar to perceptron)



All hidden nodes will have and
keep the same value!

Random Weight Initialization

Thought experiment:
Imagine all initial weights are 0
(similar to perceptron)



Random Weight Initialization

PyTorch initializes weights to small random numbers (to break symmetry) automatically for us



Next: Training a **multilayer perceptron** model in PyTorch



Defining Efficient Data Loaders

Part 1: Avoiding Data Loading Bottlenecks



For each epoch:
For each batch:

Load data

x, y



For each epoch:
For each batch:

Load data

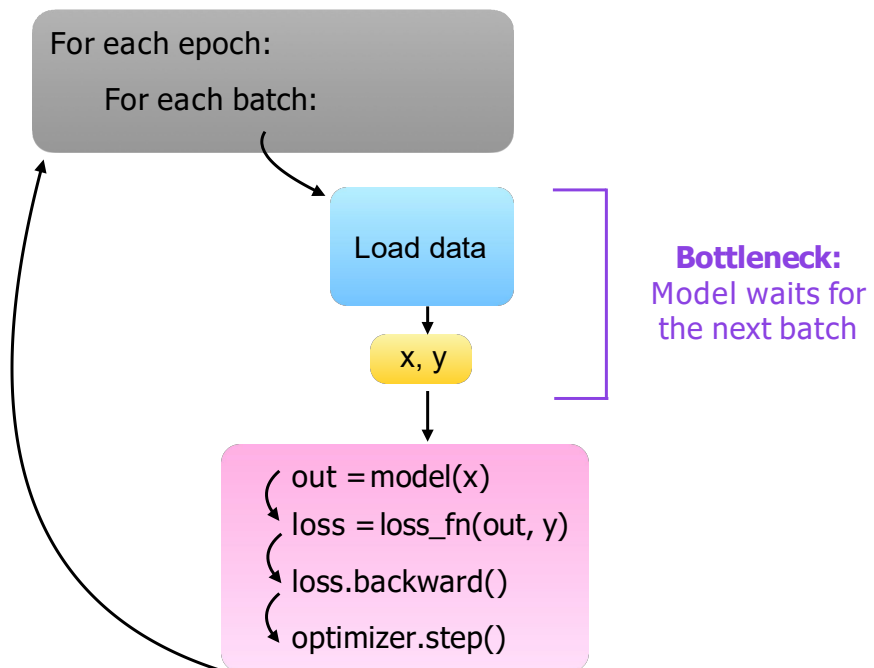
x, y

out = model(x)
loss = loss_fn(out, y)
loss.backward()
optimizer.step()

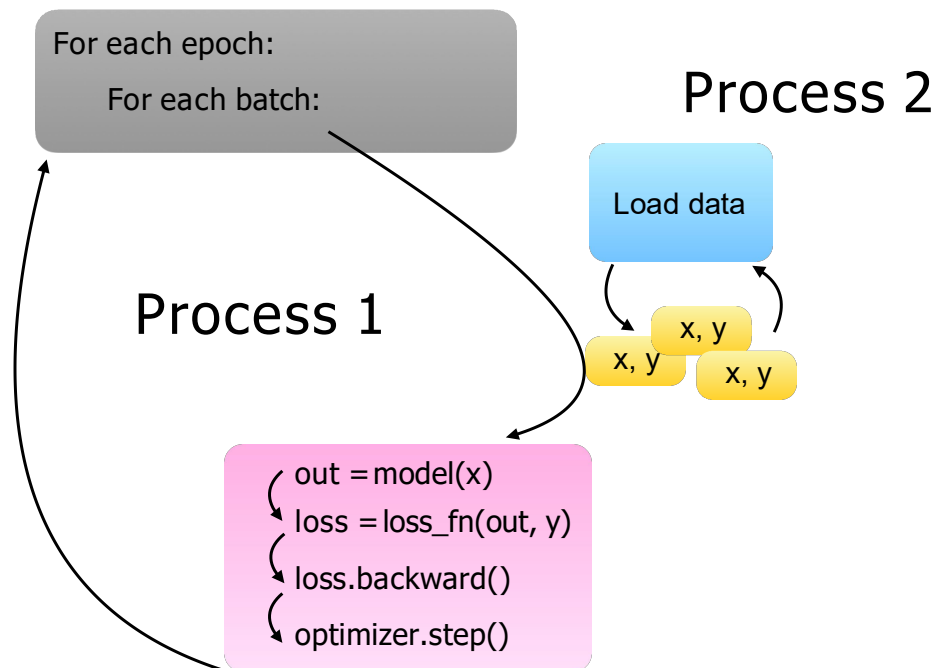


A very inefficient way to load data

```
for epoch in range(num_epochs):  
    for next_minibatch ...:  
        inputs = []  
        # prepare next minibatch  
        for f in filename_minibatch:  
            img = Image.open(os.path.join(img_dir, f))  
            img_tensor = to_tensor(img)  
            inputs.append(img_tensor)  
        inputs = torch.cat(inputs)  
  
        # train model  
        logits = model(inputs)  
        ...  
        loss.backward()  
        optimizer.step()
```



Run **data loading** and **model training**
in **separate processes** to always have
the next batch ready



```
for epoch in range(num_epochs):  
    model = model.train()  
    for batch_idx, (inputs, labels) in enumerate(train_loader):  
        logits = model(inputs)  
        loss = F.cross_entropy(logits, labels)  
        optimizer.zero_grad()  
        loss.backward()  
        optimizer.step()
```



```
from torch.utils.data import DataLoader  
  
train_loader = DataLoader(  
    dataset=train_dataset,  
    batch_size=32,  
    shuffle=True,  
    drop_last=True,  
    num_workers=2,  
)
```

Use multiple processes to
load data in the background



```
from torch.utils.data import DataLoader

train_loader = DataLoader(
    dataset=train_dataset,
    batch_size=32,
    shuffle=True,
    drop_last=True,
    num_workers=2,
)
```

Just means we drop the last batch
if training set size is not
evenly divisible by batch size



Next: DataLoader and Datasets



Defining Efficient Data Loaders

Part 2: Datasets and DataLoaders



How do **Datasets** and **DataLoaders** work together?



How do **Datasets** and **DataLoaders** work together?

Step 1: Create Dataset

Step 2: Instantiate Datasets

Step 3: Instantiate DataLoaders

Step 4: Test/Use DataLoaders



Step 1: Create a custom Dataset class

- Defines how data is loaded
- Has `__getitem__` method

Custom
Dataset class



```

import os
import pandas as pd
from PIL import Image
from torch.utils.data import Dataset

class MyDataset(Dataset):
    def __init__(self, csv_path, img_dir, transform=None):

        df = pd.read_csv(csv_path)
        self.img_dir = img_dir
        self.transform = transform

        # based on DataFrame columns
        self.img_names = df["filepath"]
        self.labels = df["label"]

    def __getitem__(self, index):
        img = Image.open(os.path.join(self.img_dir, self.img_names[index]))

        if self.transform is not None:
            img = self.transform(img)

        label = self.labels[index]
        return img, label

    def __len__(self):

```



```

import os
import pandas as pd
from PIL import Image
from torch.utils.data import Dataset

class MyDataset(Dataset):
    def __init__(self, csv_path, img_dir, transform=None):

        df = pd.read_csv(csv_path)
        self.img_dir = img_dir
        self.transform = transform

        # based on DataFrame columns
        self.img_names = df["filepath"]
        self.labels = df["label"]

    def __getitem__(self, index):
        img = Image.open(os.path.join(self.img_dir, self.img_names[index]))

        if self.transform is not None:
            img = self.transform(img)

        label = self.labels[index]
        return img, label

    def __len__(self):

```

Set up attributes



```
import os
import pandas as pd
from PIL import Image
from torch.utils.data import Dataset
```

```
class MyDataset(Dataset):
    def __init__(self, csv_path, img_dir, transform=None):
```

```
        df = pd.read_csv(csv_path)
        self.img_dir = img_dir
        self.transform = transform

        # based on DataFrame columns
        self.img_names = df["filepath"]
        self.labels = df["label"]
```

Define how to load a single record
(training example, test example)

```
    def __getitem__(self, index):
        img = Image.open(os.path.join(self.img_dir, self.img_names[index]))

        if self.transform is not None:
            img = self.transform(img)

        label = self.labels[index]
        return img, label
```

```
    def len(self):
```



```
import os
import pandas as pd
from PIL import Image
from torch.utils.data import Dataset
```

```
class MyDataset(Dataset):
    def __init__(self, csv_path, img_dir, transform=None):
```

```
        df = pd.read_csv(csv_path)
        self.img_dir = img_dir
        self.transform = transform

        # based on DataFrame columns
        self.img_names = df["filepath"]
        self.labels = df["label"]
```

```
    def __getitem__(self, index):
        img = Image.open(os.path.join(self.img_dir, self.img_names[index]))

        if self.transform is not None:
            img = self.transform(img)

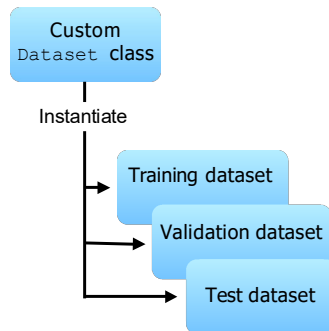
        label = self.labels[index]
        return img, label
```

```
    def len(self):
```



Step 2: Instantiate Datasets

- Defines how data is loaded
- Has `__getitem__` method



Step 2: Instantiate Datasets

```
train_dataset = MyDataset(  
    csv_path="mnist-pngs/new_train.csv",  
    img_dir="mnist-pngs/",  
    transform=data_transforms["train"],  
)  
  
val_dataset = MyDataset(  
    csv_path="mnist-pngs/new_val.csv",  
    img_dir="mnist-pngs/",  
    transform=data_transforms["test"],  
)  
  
test_dataset = MyDataset(  
    csv_path="mnist-pngs/test.csv",  
    img_dir="mnist-pngs/",  
    transform=data_transforms["test"],  
)
```



Step 3: Instantiate DataLoaders

- Defines how data is loaded
- Has `__getitem__` method

Custom
Dataset class

Instantiate

Training dataset
Validation dataset
Test dataset

- Parent class imported from PyTorch

Dataloader class

Instantiate

Train dataloader
Validation dataloader
Test dataloader

- Define
 - shuffling
 - batch size
 - number of processes
 - and more



Step 3: Instantiate DataLoaders

```
from torch.utils.data import DataLoader

train_loader = DataLoader(
    dataset=train_dataset,
    batch_size=32,
    shuffle=True,
    drop_last=True,
    transform=data_transforms["train"],
    num_workers=2,
)

val_loader = DataLoader(
    dataset=val_dataset,
    batch_size=32,
    shuffle=False,
    transform=data_transforms["test"],
    num_workers=2,
)

test_loader = DataLoader(
    dataset=test_dataset,
    batch_size=32,
    shuffle=False,
    transform=data_transforms["test"],
    num_workers=2,
)
```



Step 3: Instantiate DataLoaders

Using the
DataLoader class

```
from torch.utils.data import DataLoader

train_loader = DataLoader(
    dataset=train_dataset,
    batch_size=32,
    shuffle=True,
    drop_last=True,
    transform=data_transforms["train"],
    num_workers=2,
)

val_loader = DataLoader(
    dataset=val_dataset,
    batch_size=32,
    shuffle=False,
    transform=data_transforms["test"],
    num_workers=2,
)

test_loader = DataLoader(
    dataset=test_dataset,
    batch_size=32,
    shuffle=False,
    transform=data_transforms["test"],
    num_workers=2,
)
```



Step 3: Instantiate DataLoaders

Receiving the
dataset instances

```
from torch.utils.data import DataLoader

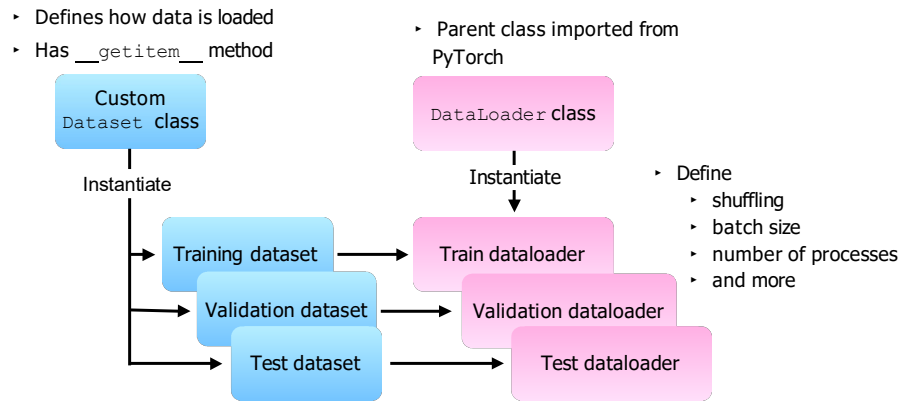
train_loader = DataLoader(
    dataset=train_dataset,
    batch_size=32,
    shuffle=True,
    drop_last=True,
    transform=data_transforms["train"],
    num_workers=2,
)

val_loader = DataLoader(
    dataset=val_dataset,
    batch_size=32,
    shuffle=False,
    transform=data_transforms["test"],
    num_workers=2,
)

test_loader = DataLoader(
    dataset=test_dataset,
    batch_size=32,
    shuffle=False,
    transform=data_transforms["test"],
    num_workers=2,
)
```



Step 4: Testing/Using the DataLoaders



Step 4: Testing/Using the DataLoaders

```
num_epochs = 1
for epoch in range(num_epochs):

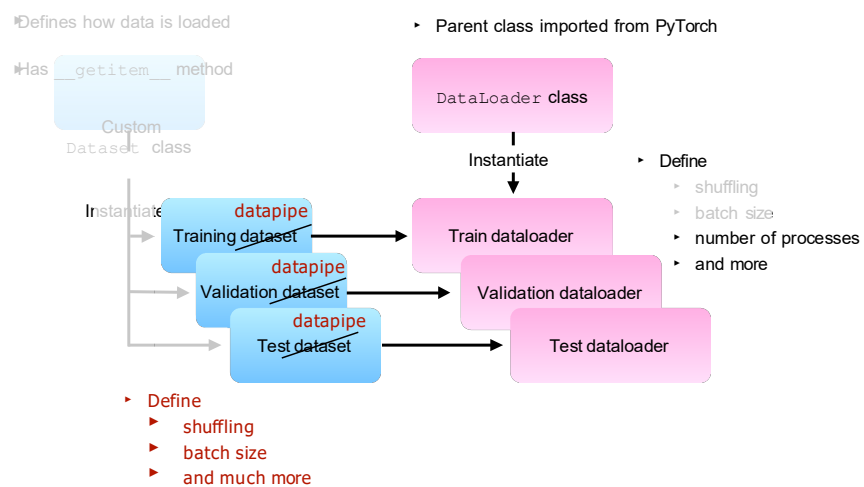
    for batch_idx, (x, y) in enumerate(train_loader):
        if batch_idx >= 3:
            break
        print(" Batch index:", batch_idx, end="")
        print(" | Batch size:", y.shape[0], end="")
        print(" | x shape:", x.shape, end="")
        print(" | y shape:", y.shape)

    print("Labels from current batch:", y)

    Batch index: 0 | Batch size: 32 | x shape: torch.Size([32, 1, 28, 28])
    | y shape: torch.Size([32])
    Batch index: 1 | Batch size: 32 | x shape: torch.Size([32, 1, 28, 28])
    | y shape: torch.Size([32])
    Batch index: 2 | Batch size: 32 | x shape: torch.Size([32, 1, 28, 28])
    | y shape: torch.Size([32])
    Labels from current batch: tensor([6, 8, 1, 6, 0, 1, 3, 8, 5, 5, 1, 2, 2
```



Outlook: Composable DataPipes



Next: Code example using Datasets and DataLoaders



Multilayer Neural Networks For Regression

Part 1: Architecture And Loss Function

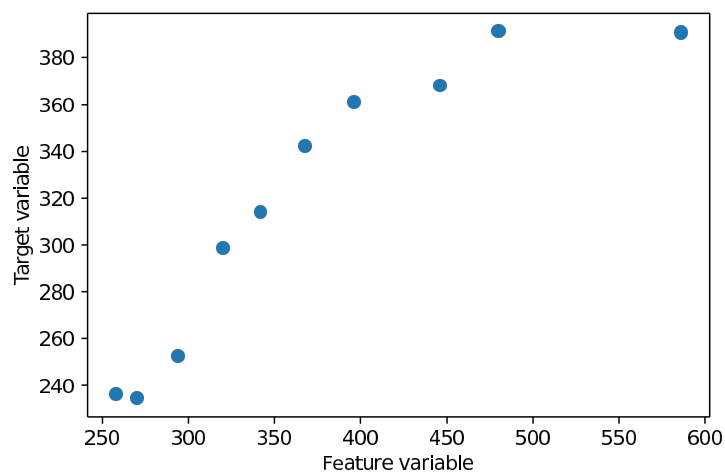


Can you use multilayer neural networks for regression?

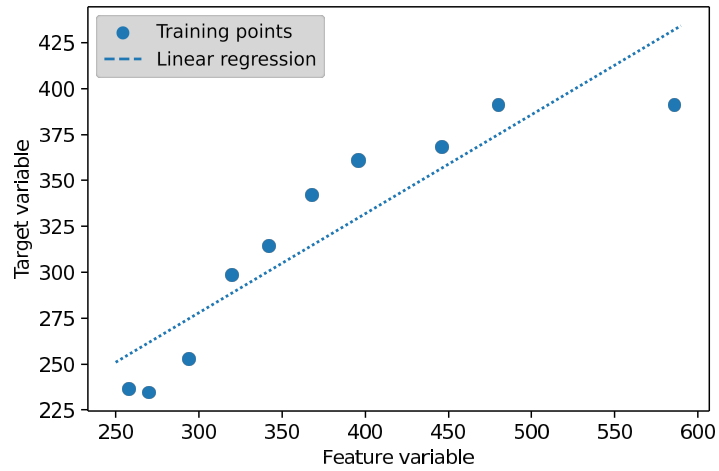
Yes, you can



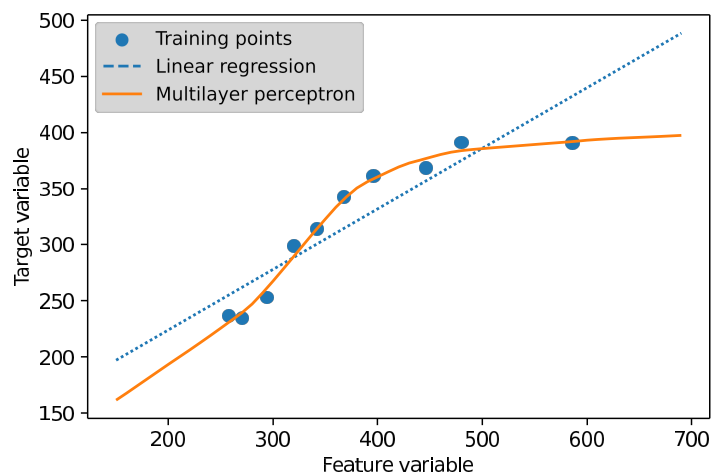
A Simple Regression Dataset



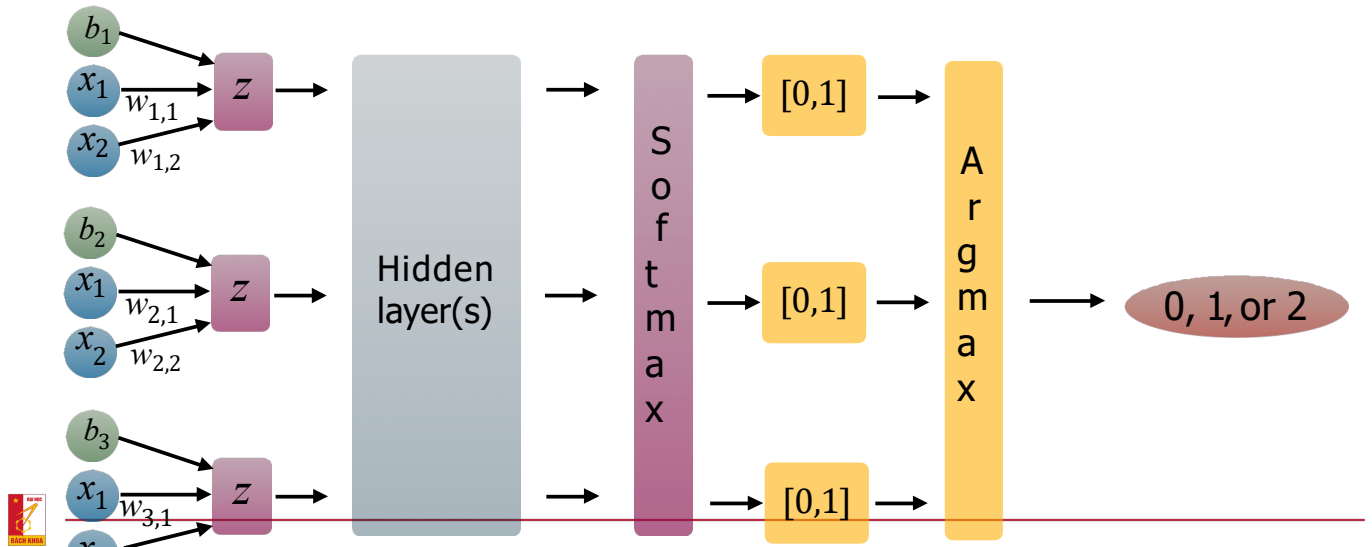
A Simple Regression Dataset



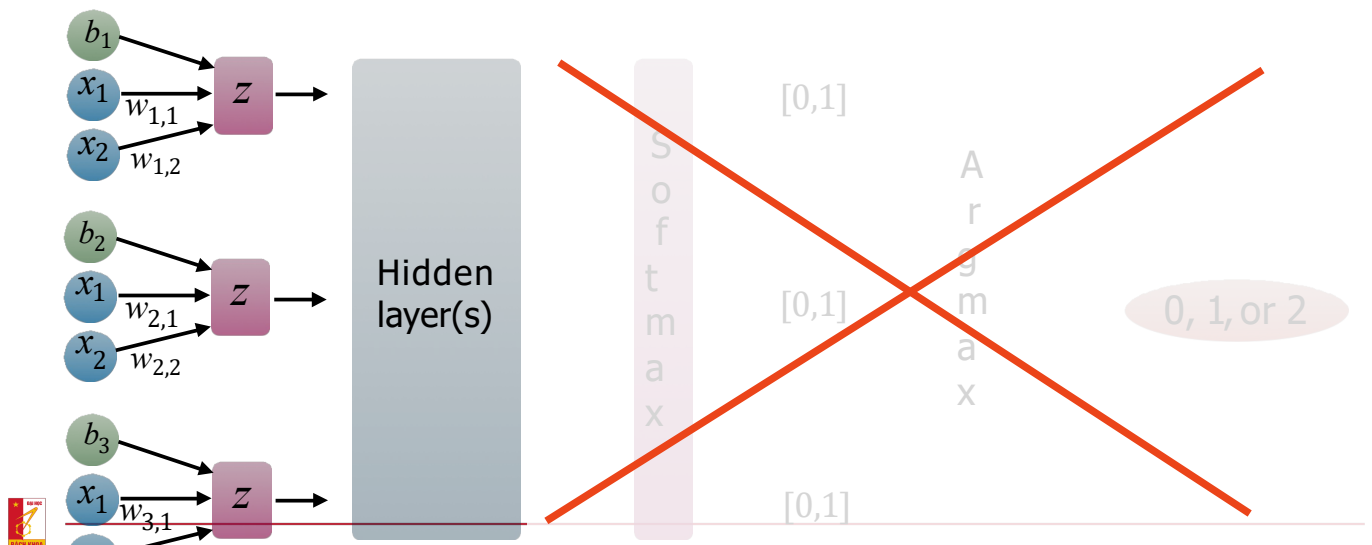
A Simple Regression Dataset



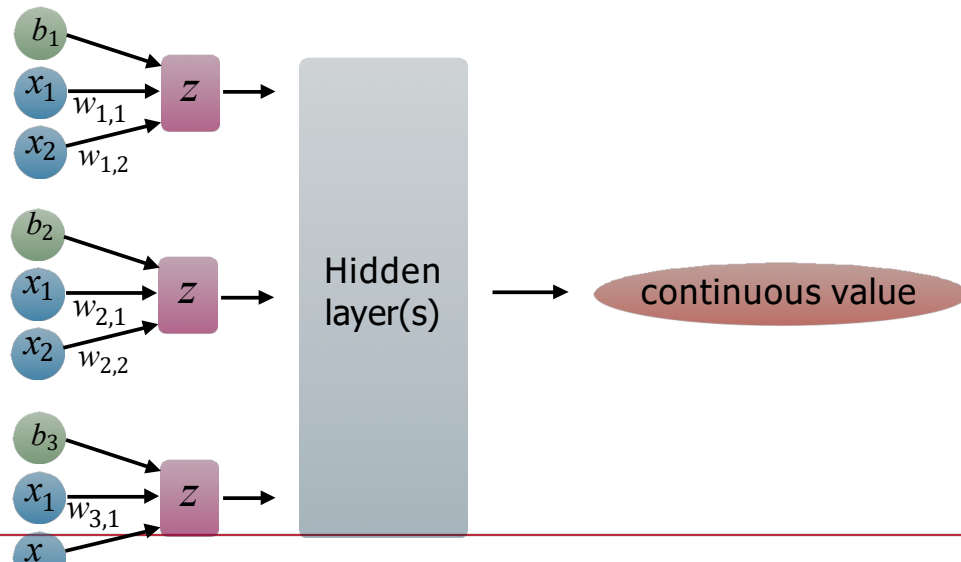
Multilayer Perceptron Model For Classification



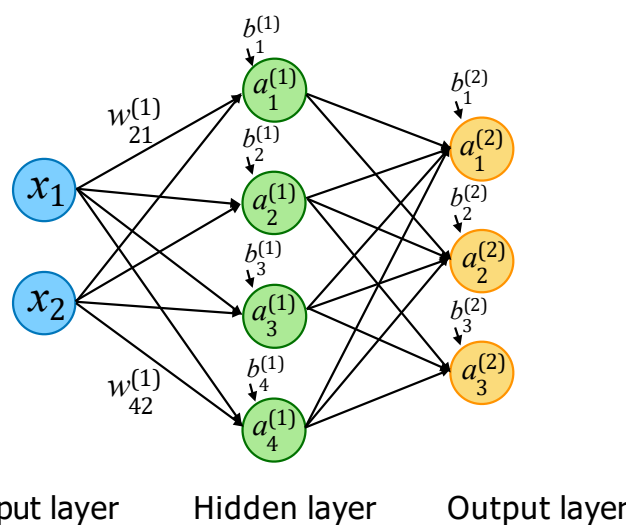
Multilayer Perceptron Model For Classification



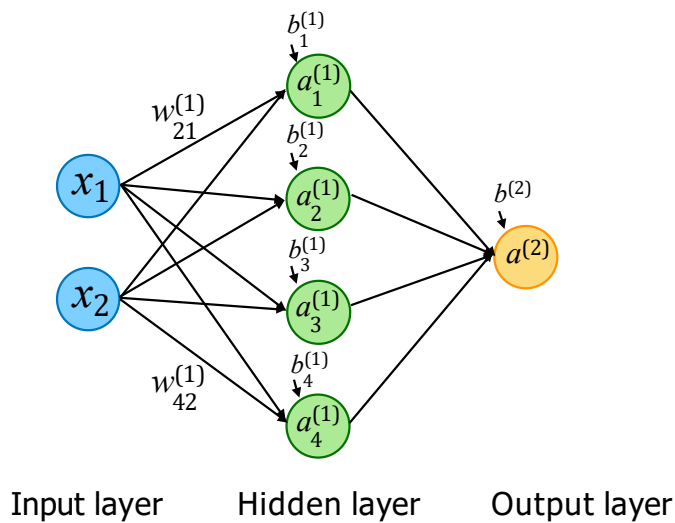
Multilayer Perceptron Model For Regression



Multilayer Perceptron Model For Classification



Multilayer Perceptron Model For Regression



Loss Function

Mean Squared Error

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y^{[i]} - \hat{y}^{[i]})^2$$

Loss Function

Mean Squared Error

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N \underbrace{(y^{[i]} - \hat{y}^{[i]})^2}_{\text{Squared difference between actual and predicted value}}$$

Squared difference
between actual and
predicted value



Loss Function

Mean Squared Error

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y^{[i]} - \hat{y}^{[i]})^2$$

Averaged over training examples

Averaged over
training examples



Next: Code Example



Speeding Up Model Training Using GPUs



1 **Tensor library**

2 **Automatic
differentiation engine**

Unit 2: What is PyTorch?

3 **Deep learning
library**



Tensors come with GPU support!

1 **Tensor library**

2 **Automatic
differentiation engine**

Unit 2: What is PyTorch?

3 **Deep learning
library**



Tensor library

`torch.tensor` `numpy.array`



Tensor library

`torch.tensor` `numpy.array`

+GPU support `a.to('cuda')`



Key idea:

Load data & model into GPU memory
for faster computing
(matrix multiplication, convolutions, etc.)



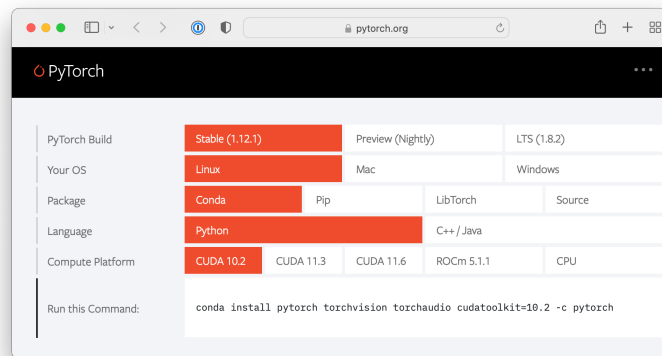
Requirements:

Computer (local or cloud)
should have a GPU

(NVIDIA, AMD, Apple Silicon, etc.)



Install a GPU Version of PyTorch



Check NVIDIA GPUs (most common) via nvidia-smi

```
current_cv_sebastianraschka_git -- raschka@lambda-quad: ~ --
(base) raschka@lambda-quad:~$ nvidia-smi
Tue Aug 9 15:18:52 2022

+-----+
| NVIDIA-SMI 455.23.05   Driver Version: 455.23.05   CUDA Version: 11.1   |
+-----+
| GPU  Name           Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|-----+-----+
| 0  GeForce GTX 108...    Off   | 00000000:05:00:0 | Off |          0%  Default |
| 30%   52C   P8      14W / 250W  |  2MiB / 11176MiB |           | N/A |
|-----+-----+
| 1  GeForce GTX 108...    Off   | 00000000:06:00:0 | Off |          0%  Default |
| 30%   52C   P8      12W / 250W  |  2MiB / 11178MiB |           | N/A |
|-----+-----+
| 2  GeForce GTX 108...    Off   | 00000000:09:00:0 | Off |          0%  Default |
| 29%   51C   P8      13W / 250W  |  2MiB / 11179MiB |           | N/A |
|-----+-----+
| 3  GeForce GTX 108...    Off   | 00000000:0A:00:0 | Off |          0%  Default |
| 25%   42C   P8      14W / 250W  |  2MiB / 11178MiB |           | N/A |
|-----+-----+

Processes:
+-----+
| GPU   GI   CI        PID   Type   Process name                      GPU Memory |
| ID    ID                                     |            Usage              |
+-----+
No running processes found
+-----+
(base) raschka@lambda-quad:~$
```



GPUs are good for parallel computations (like matrix multiplications)

Specifications	Intel® Core™ i9-11900KB Processor	NVIDIA GeForce® RTX™ 3080 Ti
Base Clock Frequency	3.3 GHz	1.37 GHz
Cores	16 (32 threads)	10240
Memory Bandwidth	45.8 GB/s	912.1 GB/s
Floating-Point Calculations	742 GFLOPS	34.10 TFLOPS
Cost	~ \$540.00	~ \$1200.00

Source: Sebastian Raschka, Yuxi (Hayden) Liu, Vahid Mirjalili. *Machine Learning with PyTorch and Scikit-Learn*. Birmingham, UK: Packt Publishing, 2022. ISBN: 978-1801819312



Loading data onto the GPU is easy!

```
[1]: import torch
      print(torch.cuda.is_available())

[1]: True

[2]: my_tensor = torch.tensor([1., 2., 3.])
      my_tensor

[2]: tensor([1., 2., 3.])

[3]: my_tensor = my_tensor.to('cuda')
      my_tensor

[3]: tensor([1., 2., 3.], device='cuda:0')

[4]: my_tensor = my_tensor.to('cpu')
      my_tensor

[4]: tensor([1., 2., 3.])
```



Only minor code
modification required

```
import torch.nn.functional as F
torch.manual_seed(1)
model = PyTorchMLP(num_features=784, num_classes=10)
model.to("cuda")
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)
num_epochs = 10
for epoch in range(num_epochs):
    model = model.train()
    for batch_idx, (features, labels) in enumerate(train_loader):
        features = features.to("cuda")
        labels = labels.to("cuda")
        logits = model(features)
        loss = F.cross_entropy(logits, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```



GPU Cloud Resources

